

國立臺灣海洋大學

大學生暑期學習實務體驗計畫成果報告撰寫格式 (請將檔案存 PDF
檔)

國立臺灣海洋大學

大學生暑期學習實務體驗計畫成果報告(封面)

計畫類別：☒研究類、☐實作類、☐產學交流類

執行期間：113 年 7 月 1 日至 113 年 8 月 31 日

計畫名稱：物理訊息神經網路

學生姓名：張睿埕、潘陳資蕎、沈峻毅、侯力維

學生學號：0106C032、0106C036、0116C029、0116C038

學系及年級：通訊 4A、通訊 3A

王和盛

指導教授：王和盛 教授

中華民國 113 年 9 月 18 日

物理訊息神經網路

指導老師: 王和盛 教授

組員: 0106C032 張睿埕、0106C036 潘陳資蕎、0116C029 沈峻
毅、0116C038 侯力維

摘要:

本次研究專題主要探討了使用物理信息神經網路 (Physics-Informed Neural Networks, PINNs) 來解決偏微分方程 (Partial Differential Equations, PDEs) 的數個具體問題。首先, 我們求解了在三維球座標系中均勻球體電位分布的 Laplace 方程。該問題在物理學和工程領域中具有廣泛應用, 特別是在電磁場理論中。我們利用 PINNs 成功地求解了此問題, 展示了該方法在高維度問題中的強大能力。

其次, 我們在二維笛卡爾座標系上探討了 Poisson 方程的解。傳統方法在處理該方程的時候, 通常需要已知方程中的源項函數 $f(x,y)$ 的形式。然而, 在實際應用中, 源項函數 $f(x,y)$ 往往是未知的, 但整個函數未知對求解有困難, 所以我們假設 Poisson 方程中的 $f(x,y)$ 只有係數是未知, 並利用 PINN 來同時求解方程和識別未知係數。這一新方法展示了 PINN 在處理含未知參數的偏微分方程中的潛力。

再者, 我們延伸命題, 考慮一個簡單的三維無限深位能阱的問題, 利用 PINN 來模擬基態電子在位能阱中的物質波波函數的實際長相。成功使用 PINN 模擬出近似的實際波函數解。

實驗結果表明, PINN 能夠在不需要大量數值求解的情況下, 精確地求解拉普拉斯和 Poisson 方程, 並有效地識別未知參數。這表明 PINN 在工程和科學計算中的應用前景廣闊, 尤其在處理複雜邊界條件和高維度問題時具有顯著優勢。研究結果也為未來在更廣泛的物理和工程應用中利用 PINNs 提供了有力的證據和參考。

目錄

● 摘要-----	2
● 前言-----	4
● 研究動機與目的-----	4
● 研究材料與方法:	
■ Laplace 方程-----	5
■ Poisson 方程-----	10
■ 無限位能量子阱-----	20
● 結果:	
■ Laplace 方程-----	26
■ Poisson 方程-----	28
■ 無限位能量子阱-----	39
● 結論與建議-----	40
● 參考文獻-----	41
● 暑期計畫成果自評-----	42
● 指導教授評語-----	42
● 附件:	
■ 程式碼-----	42

前言

近年來，隨著深度學習技術的快速發展，物理信息神經網絡（Physics-Informed Neural Networks, PINNs）作為一種新興的數值求解方法，逐漸引起了廣泛關注。PINN 通過將物理定律嵌入神經網絡的結構中，不僅能夠有效地求解偏微分方程（PDEs），還可以在一定程度上克服傳統方法的限制。相比傳統的數值求解方法，如有限差分法、有限元法等，PINN 在處理含有未知參數的問題和高維度問題上展現出了顯著的優勢。

物理信息神經網絡的核心思想是利用深度神經網絡來逼近 PDE 的解，並通過將物理約束（如 PDE 方程）和初始條件、邊界條件直接嵌入到損失函數中。PDE 約束部分被稱為 PDE loss，而初始條件或邊界條件則作為 data loss 的一部分。這種方法不僅能夠提高求解精度，還能夠減少對數據的需求，特別是在實際應用中往往難以獲取大量數據的情況下，PINN 展現出了極大的潛力。

研究動機與目的

本研究的動機來源於 PINN 在數值求解領域的潛力及其在實際應用中的優勢。我們希望通過實驗 PINN 在解決具體物理問題中的表現，來進一步驗證其有效性和實用性。具體而言，本研究聚焦於以下三個具體問題：

1. 三維球座標中均勻球體電位分布的 Laplace 方程：

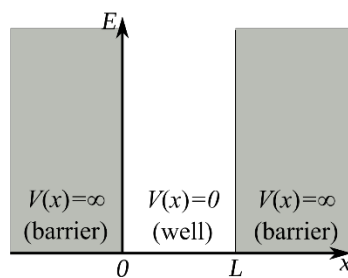
該問題在電磁場理論中具有廣泛的應用，對於理解許多物理現象至關重要。我們假設使用 Dirichlet 邊界條件來約束邊界上的電位值，求解該 Dirichlet 問題。通過這個例子，我們希望驗證 PINN 在解決三維球座標問題中的有效性。

2. 二維笛卡爾座標系上的 Poisson 方程：

在實際應用中，Poisson 方程中的源項函數 $f(x, y)$ 通常是未知的，但這樣的未知假設太過困難，因此，我們改為假設其係數未知，並利用 PINN 來同時求解方程和識別未知係數。我們將分別假設 Dirichlet、Neumann 和 Robin 三種邊界條件進行比較，並求解方程問題。通過這個例子，我們希望展示 PINN 在處理含有未知參數的問題，以及對不同邊界條件的適應性。

3. 無限位能量子阱中的波函數：

無限位能量子阱是經典的量子物理學命題，在這個問題中，波函數量值的絕對值平方相當於粒子在量子阱中出現的機率，粒子被限制在一個形狀為立方體的有限區域內，在立方體的邊界以及外部，波函數量值恆為零。波函數實際上為薛丁格方程式，並利用 Dirichlet 邊界條件限制粒子在立方體邊界上的波函數。此題目考慮一個三維立方體區域，邊長分別為 L_x 、 L_y 、 L_z ，假設有一粒子在此區域內，求解其物質波之波函數。

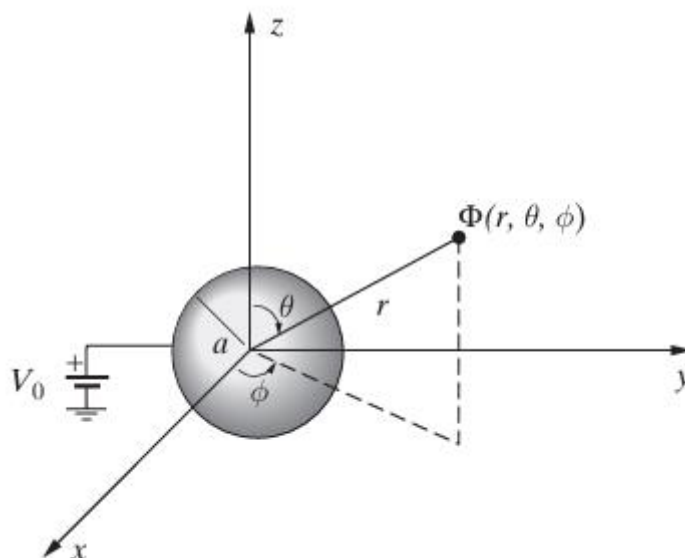


本研究的目的是在於通過對上述兩個具體問題的探討，驗證 PINN 在解決不同類型物理問題中的可行性和優越性，並為未來在其他物理問題中的應用提供理論基礎和實踐參考。

研究材料與方法

1. Laplace 方程

(問題描述)考慮一個如圖所示經由電池連接而被維持在電位 $\Phi(a, \theta, \varphi) = v_0$ 的球形導體，求解 $\Phi(r, \theta, \varphi)$ 。假設球體周圍的區域是真空，且附近沒有其他電荷或導體存在，因此當 $r \rightarrow \infty$ 時 $v \rightarrow 0$ 。



首先因為我們要求解的是靜電平衡狀態下無電荷分佈區域內的電勢分佈，這符合 Laplace 方程的使用條件。在靜電平衡狀態下，導體表面上的電勢是均勻的，且其周圍區域無其他電荷或導體存在，因此電勢分佈滿足 Laplace 方程 $\Delta\Phi = 0$ 。

而 Laplace 方程在球面座標系上的定義如下

$$\Delta\Phi = \frac{1}{r^2} \frac{\partial}{\partial r} \left(r^2 \frac{\partial \Phi}{\partial r} \right) + \frac{1}{r^2 \sin\theta} \frac{\partial}{\partial \theta} \left(\sin\theta \frac{\partial \Phi}{\partial \theta} \right) + \frac{1}{r^2 \sin^2\theta} \frac{\partial^2 \Phi}{\partial \varphi^2} = 0$$

接著因為球對稱性質，電位 Φ 不會是包含 θ 、 φ 的函數，電位 Φ 只跟 r 有關，故第二項與第三項為零，所以第一項也為零。

$$\frac{1}{r^2} \frac{\partial}{\partial r} \left(r^2 \frac{\partial \Phi}{\partial r} \right) = 0 \Rightarrow \frac{\partial}{\partial r} \left(r^2 \frac{\partial \Phi}{\partial r} \right) = 0$$

$$r^2 \frac{\partial \Phi}{\partial r} = C_1 \Rightarrow \Phi(r) = -\frac{C_1}{r} + C_2$$

然後將邊界條件 $\Phi(a, \theta, \varphi) = v_0$ 帶入 $\Phi(r) = -\frac{C_1}{r} + C_2$ 可得

$$v_0 = -\frac{C_1}{a} + C_2 \Rightarrow C_1 = -v_0 a + C_2 a$$

帶入 $\Phi(r) = -\frac{C_1}{r} + C_2$ 整理得

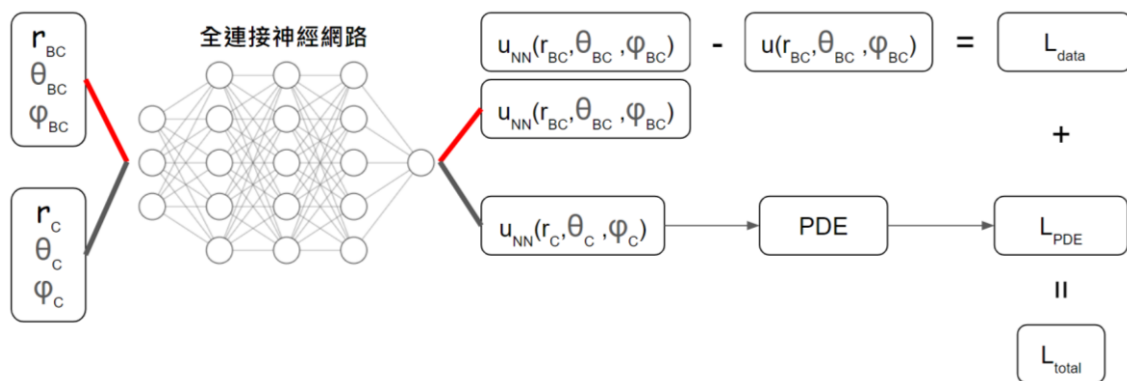
$$\Phi(r) = \frac{V_0 a}{r} + C_2 \left(1 - \frac{a}{r} \right)$$

然後在將邊界條件 $\Phi(\infty, \theta, \varphi) = 0$ 帶入 $\Phi(r) = \frac{V_0 a}{r} + C_2 \left(1 - \frac{a}{r} \right)$ 可得球外的電勢分佈為

$$\Phi(r) = \frac{V_0 a}{r}$$

為了使用 PINN 來求解此問題，我們將 Laplace 方程和 Dirichlet 邊界條件嵌入到神經網絡的損失函數中。這樣，神經網絡不僅學習數據驅動的模式，還遵循物理方程，從而提高預測的精度和可靠性。PINN 的核心思想是通過將物理方程作為約束條件融入損失函數，來引導神經網絡的學習過程。

具體架構如下：



首先，我們會生成許多隨機且離散的邊界資料點與內部資料點。接著，將邊界資料點帶入之前推得的解析解 $\Phi(r) = \frac{V_0 a}{r}$ ，以求出正確的邊界值(Dirichlet BC)。這一步驟是為了模擬量測時得到的邊界資料點的邊界值，或者將一些已知的邊界條件函數轉為隨機離散的資料點，實際上解析解並無參與訓練。然後，我們將邊界資料點帶入神經網絡，並將神經網絡輸出與已知的正確邊界值做均方誤差運算，得到 data loss。

接著，將內部資料點帶入神經網絡，並將神經網絡輸出再帶入 Laplace 方程中。根據定義，Laplace 方程的右側為零，因此若 Laplace 方程（PDE 格子）的輸出不為零時，這一部分就被定義為 PDE loss。接著，我們將 PDE loss 與 data loss 相加，得到 total loss。

最後，通過反向傳播和梯度下降算法來更新神經元權重，使 total loss 最小化並收斂，此時神經網絡的輸出就會很逼近解析解。

具體步驟如下：

1. 定義精確解。

```
def true_solution(r, theta, phi):
    return v0*a/r
```

此處我們假設 $v_0 = 5$ 、 $a = 1$ ，也就是將半徑為 1 的球面電位維持在 5V。

2. 生成半徑為 1 的 300 個隨機邊界資料點與半徑為 100 的 300 個隨機邊界資料點，並帶入精確解求 Dirichlet BC、儲存。

```

for i, j in zip(range(n_bc), [a, max_r]):
    points = engine.random(n=n_data_per_bc)
    points[:, 0] = points[:, 0]*np.pi
    points[:, 1] = points[:, 1]*2*np.pi

    data[i, :, 0] = j
    data[i, :, 1] = points[:, 0]
    data[i, :, 2] = points[:, 1]

```

```

for i in range(n_bc):
    for j in range(n_data_per_bc):
        data[i, j, 3] = true_solution(data[i, j, 0], data[i, j, 1], data[i, j, 2])

```

3. 生成半徑為 1~100 內的 600 個內部資料點，並儲存。

```

colloc[:, 0] = colloc[:, 0]*(max_r-a)+a
colloc[:, 1] = colloc[:, 1]*np.pi
colloc[:, 2] = colloc[:, 2]*2*np.pi

```

4. 定義並構建一個全連接神經網絡函數，該網絡包含 3 個輸入、1 個輸出、6 層隱藏層，每層隱藏層有 25 個神經元，並使用雙曲正切 (tanh) 函數作為激活函數。

```

def DNN_builder(in_shape=3, out_shape=1, n_hidden_layers=5, neuron_per_layer=20, actfn="tanh"):
    #使用 Keras 函數式 API 建立一個具有指定架構的全連接神經網路模型
    # input layer 輸入層
    input_layer = tf.keras.layers.Input(shape=(in_shape,))
    # hidden layers 隱藏層
    hidden = [tf.keras.layers.Dense(neuron_per_layer, activation=actfn)(input_layer)]
    for i in range(n_hidden_layers-1):
        new_layer = tf.keras.layers.Dense(neuron_per_layer, activation=actfn, activity_regularizer=None)(hidden[-1])
        hidden.append(new_layer)
    # output layer
    output_layer = tf.keras.layers.Dense(1, activation=None)(hidden[-1])
    # building the model
    name = f"DNN-{n_hidden_layers}"
    model = tf.keras.Model(input_layer, output_layer, name=name)
    return model

```

```

model = DNN_builder(3, 1, 6, 25, "tanh")##ReLU tanh

```

此處我們利用 TensorFlow 中的 Keras 模塊來構建一個全連接神經網絡。

5. 定義神經網路輸出 $u_{nn}(r, \theta, \varphi)$

```

@tf.function
def u(r, theta, phi):
    u = model(tf.concat([r, theta, phi], axis=1))
    return u

```

6. 定義 Laplace equation，並對輸出做均方误差運算


```

@tf.function
def f(r, theta, phi):
    u0 = u(r, theta, phi)
    #偏微分方程(極座標形式)
    u_r = tf.gradients(u0, r)[0]
    u_theta = tf.gradients(u0, theta)[0]
    u_phi = tf.gradients(u0, phi)[0]

    u_rr = tf.gradients(u_r*r*r, r)[0]
    u_tt = tf.gradients(u_theta*tf.sin(theta), theta)[0]
    u_pp = tf.gradients(u_phi, phi)[0]

    # F = (1/(r*r))*u_rr + (1/(r*r*tf.sin(theta)))*u_tt + (1/(r*r*tf.sin(theta)*tf.sin(theta)))*u_pp
    F = tf.sin(theta)*tf.sin(theta)*u_rr + tf.sin(theta)*u_tt + u_pp

    return tf.reduce_mean(tf.square(F))

```

此處因為 r 介於 $1 \sim 100$ 間，當乘上 $1/r^2$ 時會讓 PDE loss 隨半徑變大而自動減小很多，為防止此狀況影響訓練效果，我們做一個預處理，將 Laplace equation 乘上 r^2 ，變成 $\Delta\Phi = 0 \Rightarrow (\Delta\Phi) \times r^2 = 0 \times r^2 = 0$ 。另外 $1/\sin(\theta)$ 當 $\theta = 0$ or 180 時，會發生除到零的情況，所以一樣需要在對式子乘上 $\sin(\theta)^2$ ，做完所有預處理後，PDE 變為

$$\Delta\Phi = \sin^2\theta \frac{\partial}{\partial r} \left(r^2 \frac{\partial\Phi}{\partial r} \right) + \sin\theta \frac{\partial}{\partial\theta} \left(\sin\theta \frac{\partial\Phi}{\partial\theta} \right) + \frac{\partial^2\Phi}{\partial\varphi^2} = 0$$

7. 定義均方误差

```

@tf.function
def mse(val, val_):
    return tf.reduce_mean(tf.square(val-val_))

```

8. 設定神經網路的優化器為 Adam，並用餘弦衰減動態調整學習率

```

# 定义初始学习率和衰减步骤数
initial_learning_rate = 1e-3
decay_steps = epochs
alpha = 0
# 使用余弦衰减学习率调度器
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=initial_learning_rate, decay_steps=decay_steps, alpha=alpha)
opt = tf.keras.optimizers.Adam(learning_rate=lr_schedule)

```

9. 執行 100000 的迭代訓練

```

# 訓練迴圈
for epoch in range(epochs):
    # 使用 GradientTape 計算損失函數的梯度
    with tf.GradientTape() as tape:
        T_ = u(r_d, theta_d, phi_d) #將邊界資訊(x,y)丟給神經網路 得到預測輸出值
        # loss on physics - PDE
        #將邊界內的點代入神經網路得到輸出u 在代入計算偏微分方程
        #因為Laplace方程Δu=0(輸出值因為該為0) 但實際上輸出會因為輸入u的好壞而影響不會剛好是0 故此項數值定義為PDF誤差
        L = f(r_c, theta_c, phi_c)
        # loss on data
        # 將邊界條件(c) (c是儲存邊界條件的解 ex: u(1,y)=1 ) 與預測輸出值 相減 得到data誤差 (??邊界誤差??)
        l = mae(t_d, T_)
        # Add the loss on data and loss on equation ....
        # 將兩種誤差相加成總誤差
        loss = lambda1*L + lambda2*l

    # 計算總損失對模型參數的梯度
    g = tape.gradient(loss, model.trainable_weights)
    # 使用優化器更新模型參數
    opt.apply_gradients(zip(g, model.trainable_weights))

    if epoch % 100 == 0 or epoch == epochs-1: # 每隔 100 個 epoch 或最後一個 epoch 記錄當前的損失值
        print(f"[epoch:5], {loss.numpy():.9f}")
        loss_values = np.append(loss_values, loss)
        L_values = np.append(L_values, L)
        l_values = np.append(l_values, l)

```

在實作時，我們有發現到模型在邊界處的訓練效果很好，但在邊界內的誤差就有點大，為此我們調整了 total loss 的計算，給予了 PDE loss 較高的權重($\lambda_1 = 10^5$ 、 $\lambda_2 = 1$)，來改善這一情況。

10. 完整程式碼在附件處

2. Poisson 方程

已知在二維笛卡爾座標系中，Poisson 方程的定義為：

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \Rightarrow \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - f(x, y) = 0$$

其中 Δ 表示拉普拉斯算子， $f(x, y)$ 是源項函數。在實際應用中， $f(x, y)$ 通常是未知的，但假設整個函數未知可能過於困難，因此，我們假設其係數未知，並利用 PINN 來同時求解方程並識別未知係數。

此處我們舉兩個符合 Poisson 方程的例子來進行實驗

(a) $f(x, y) = 4(x^2 + y^2 + 1)e^{x^2+y^2}$ 、 $u(x, y) = e^{x^2+y^2}$

(b) $f(x, y) = e^{xy}(x^2 + y^2) + \sinh(x)$ 、 $u(x, y) = e^{xy} + \sinh(x)$

我們假設並不知道原項函數的係數與解析解 u ，此時我們的問題描述為

(a) $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - k_0(x^2 + y^2 + 1)e^{x^2+y^2} = 0$ ，求 $u(x, y)$ 、 k_0

(b) $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - k_0 e^{xy}(x^2 + y^2) - k_1 \sinh(x) = 0$ ，求 $u(x, y)$ 、 k_0 、 k_1

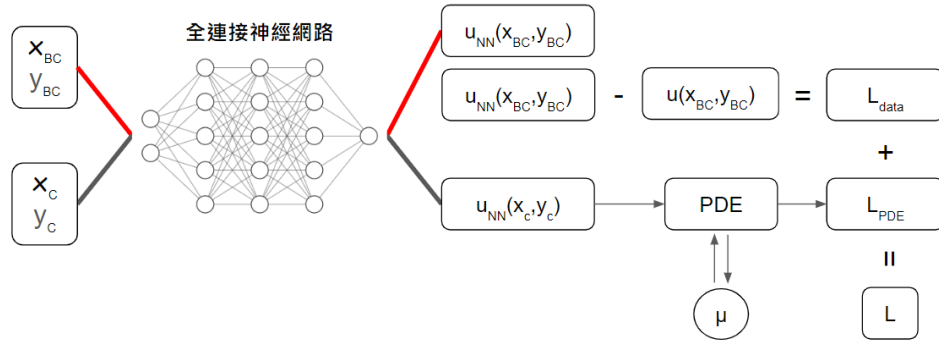
設 x 和 y 都是介於0~1之間的範圍，且分別假設已知 Dirichlet、

Neumann、Robin 三個邊界條件。

為了使用 PINN 來求解此問題，我們將 Poisson 方程和邊界條件嵌入到神經

網絡的損失函數中，詳細的 PINN 架構如下

1. PINN 解 Poisson equations 與參數的架構(Dirichlet BC)



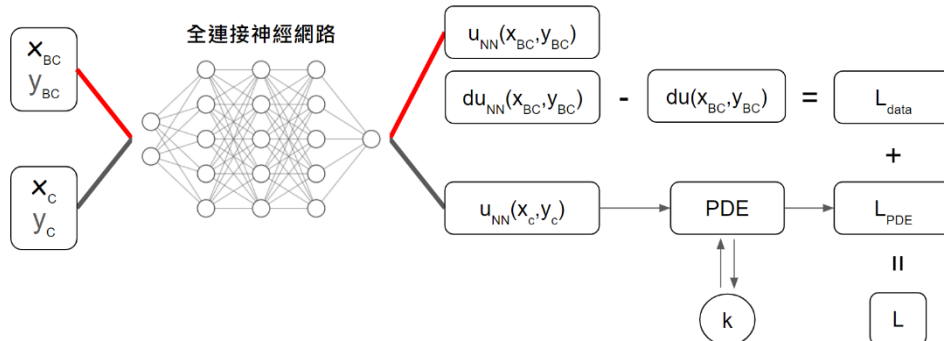
首先跟之前一樣，會先產生許多隨機且離散的邊界資料點與內部資料點，接著會使用解析解計算邊界資料點的邊界值，得到 Dirichlet BC，由於邊界條件是已知的，因此解析解並無參與到訓練中。

然後是 data loss 的計算，我們將邊界資料點帶入神經網絡，並將神經網絡輸出與已知的 Dirichlet BC 做均方誤差運算，得到 data loss。

接著是 PDE loss 的計算，這裡我們將原項函數改寫為 $f(x, y, \mu)$ ，新增一個可學習參數 μ ，用於表達 $f(x, y)$ 中的未知係數項，然後將內部資料點帶入神經網絡，並將神經網絡輸出再帶入 Poisson 方程中，根據定義 $\Delta u = f(x, y, \mu)$ ，經過移項可得 $\Delta u - f(x, y, \mu) = 0$ ，因此若方程的輸出不為零時，這一部分就被定義為 PDE loss。

之後，將 PDE loss 與 data loss 相加，得能到 total loss，通過反向傳播和梯度下降算法來更新神經元權重與可學習參數 μ ，使 total loss 最小化並收斂，此時神經網絡的輸出就會很逼近解析解， μ 也會逼近真實係數值 k 。

2. PINN 解 Poisson equations 與參數的架構(Neumann BC)

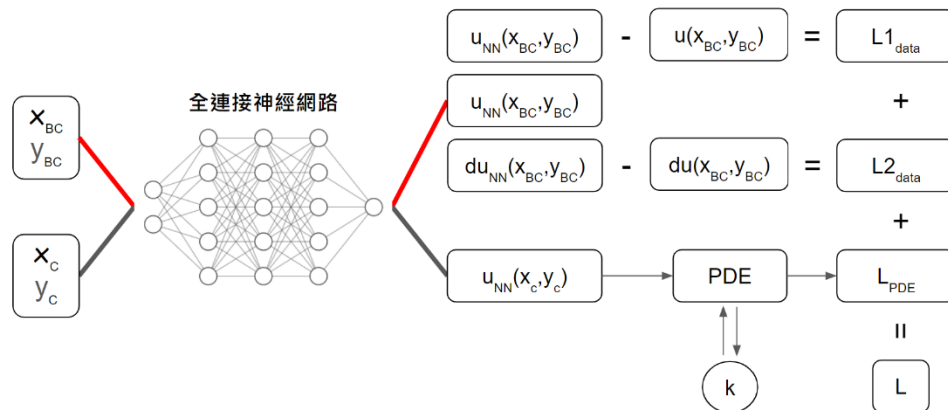


與 Dirichlet BC 架構類似，差別在於計算 data loss 時，會將神經網絡輸出做一次導數運算，然後與已知的 Neumann BC 做均方誤差，得到 data

loss。

實作上，我們會對解析解做一次偏微分，並將離散邊界資料點帶入，得到 Neumann BC，由於假設已知 Neumann BC，因此解析解並無參與到訓練中。

3. PINN 解 Poisson equations 與參數的架構(Robin BC)



此架構等於 Dirichlet BC 與 Neumann BC 架構的合併，在計算 data loss 時，會先計算兩種邊界條件的 data loss，在將兩種 data loss 合併為 Robin BC 架構下的 data loss，而 PDE loss 計算方式則維持不變。

問題(a)的具體步驟如下：

1. 定義精確解與偏微分結果

```
def tru(x, y):
    tru=np.exp(x*x+y*y)
    return tru

def trudx(x, y):
    trudx=np.exp(x*x+y*y)*2*x
    return trudx

def trudy(x, y):
    trudy=np.exp(x*x+y*y)*2*y
    return trudy
```

2. 定義源項函數 $f(x, y, \mu)$ 並設真實係數值 $k = 4$

```
k0=4
def f(x, y, mu):
    tru = mu*(x*x+y*y+1)*torch.exp(x*x+y*y)
    return tru
```

3. 定義一個全連接神經網絡

```
class FCN(nn.Module):
    "Defines a standard fully-connected network in PyTorch"

    def __init__(self, N_INPUT, N_OUTPUT, N_HIDDEN, N_LAYERS):
        super().__init__()
        activation = nn.Tanh # 激活函數
        self.fcs = nn.Sequential(*[nn.Linear(N_INPUT, N_HIDDEN), activation()])
        # 重複 N_LAYERS - 1 次, 以創建多個隱藏層
        self.fch = nn.Sequential(*[nn.Sequential(*[nn.Linear(N_HIDDEN, N_HIDDEN), activation()]) for _ in range(N_LAYERS-1)])
        self.fce = nn.Linear(N_HIDDEN, N_OUTPUT)

    def forward(self, x):
        x = self.fcs(x)
        x = self.fch(x)
        x = self.fce(x)
        return x
```

4. 生成 $x = 0$ 、 $x = 1$ 、 $y = 0$ 、 $y = 1$ 各 30 個隨機邊界資料點，並帶入精確解求 Dirichlet BC、Neumann BC。

```
for i, j in zip(range(n_bc), [0., 1., 0, 1.]):
    points = (engine.random(n=n_data_per_bc)[:, 0] - 0.) * 1

    if i < 2:
        data[i, :, 0] = j
        data[i, :, 1] = points
    else:
        data[i, :, 0] = points
        data[i, :, 1] = j
```

```
for j in range(0, n_data_per_bc):
    # bord x = 0
    # 計算Dirichlet problem 邊界問題
    data[0, j, 2] = tru(data[0, j, 0], data[0, j, 1])
    # 計算Neumann problems 邊界問題 (導數)
    data[0, j, 3] = trudx(data[0, j, 0], data[0, j, 1])
    data[0, j, 4] = trudy(data[0, j, 0], data[0, j, 1])

    # bord x = 1
    data[1, j, 2] = tru(data[1, j, 0], data[1, j, 1])
    data[1, j, 3] = trudx(data[1, j, 0], data[1, j, 1])
    data[1, j, 4] = trudy(data[1, j, 0], data[1, j, 1])

    # bord y = 0
    data[2, j, 2] = tru(data[2, j, 0], data[2, j, 1])
    data[2, j, 3] = trudx(data[2, j, 0], data[2, j, 1])
    data[2, j, 4] = trudy(data[2, j, 0], data[2, j, 1])

    # bord y = 1
    data[3, j, 2] = tru(data[3, j, 0], data[3, j, 1])
    data[3, j, 3] = trudx(data[3, j, 0], data[3, j, 1])
    data[3, j, 4] = trudy(data[3, j, 0], data[3, j, 1])
```

5. 生成 $0 \leq x \leq 1$ 、 $0 \leq y \leq 1$ 的 500 個內部資料點，並儲存。

```
 Nc = 500
engine = qmc.LatinHypercube(d=2)
colloc = engine.random(n=Nc)
colloc = 1 * (colloc - 0)
```

6. 創建一個神經網路，包括 2 個輸入、1 個輸出、6 層隱藏層，每層隱藏層有 25 個神經元。

```
pinn = FCN(2, 1, 25, 6)
```

7. 定義 PINN 輸出與偏微分結果

```
def u_nn(x, y):
    u = pinn(torch.cat([x, y], dim=1)) #排成[[x0, y0],
                                         #      [x1, y1]]
    return u

def uderx(x, y):
    u = pinn(torch.cat([x, y], dim=1))
    uderx = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
    return uderx

def udery(x, y):
    u = pinn(torch.cat([x, y], dim=1))
    udery = torch.autograd.grad(u, y, torch.ones_like(u), create_graph=True)[0]
    return udery
```

8. 定義 Poisson equations，並對輸出做均方誤差。

```
def F_nn(x, y, mu):
    u0 = u_nn(x, y)
    u_x = torch.autograd.grad(u0, x, torch.ones_like(u0), create_graph=True)[0]
    u_y = torch.autograd.grad(u0, y, torch.ones_like(u0), create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    u_yy = torch.autograd.grad(u_y, y, torch.ones_like(u_y), create_graph=True)[0]
    # Select the residual equation ....
    F = u_xx + u_yy - f(x, y, mu)
    retour = torch.mean(F ** 2)
    return retour
```

9. 定義均方誤差

```
def mse(y, y_):
    return torch.mean((y-y_)**2)
```

10. 定義了一個可學習的參數 μ (標量),用於估計原項函數中的真實係數值 k

```
mu = torch.nn.Parameter(torch.zeros(1, requires_grad=True))
```

11. 設定神經網路的優化器為 Adam 和學習率為 2×10^{-4} 、 10^{-1}

```
optimiser1 = torch.optim.Adam(pinn.parameters(), lr=1e-4)
optimiser2 = torch.optim.Adam([mu], lr=1e-1)
```

12. 執行 80000 的迭代訓練

Dirichlet BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_rnn(x_c, y_c, mu)
    #loss on data
    T_ = u_rnn(x_d, y_d)
    l = mse(t_d, T_)

    loss = L + l
    loss.backward() #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    optimiser1.step() #使用優化器的step()方法更新模型參數
    optimiser2.step()

    mus.append(mu.item())

if epoch % 100 == 0 or epoch == epochs-1:
    print(f"epoch:5], {loss.detach().numpy():.9f}")
    loss_values = np.append(loss_values, loss.detach().numpy())
    L_values = np.append(L_values, L.detach().numpy())
    l_values = np.append(l_values, l.detach().numpy())

```

Neumann BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_rnn(x_c, y_c, mu)
    #loss on data
    T_ = uderx(x_d, y_d)
    l = mse(t_dx, T_)

    T_ = uderx(x_d, y_d)
    l += mse(t_dy, T_)

    loss = L + l
    loss.backward() #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    optimiser1.step() #使用優化器的step()方法更新模型參數
    optimiser2.step()

    mus.append(mu.item())

if epoch % 100 == 0 or epoch == epochs-1:
    print(f"epoch:5], {loss.detach().numpy():.9f}")
    loss_values = np.append(loss_values, loss.detach().numpy())
    L_values = np.append(L_values, L.detach().numpy())
    l_values = np.append(l_values, l.detach().numpy())

```

Robin BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_rn(x_c, y_c, mu)
    #loss on data
    T_ = u_rn(x_d, y_d)
    l = mse(t_d, T_)

    T_ = uderx(x_d, y_d)
    l += mse(t_dx, T_)

    T_ = uderx(x_d, y_d)
    l += mse(t_dy, T_)

    loss = L + l
    loss.backward() #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    optimiser1.step() #使用優化器的step()方法更新模型參數
    optimiser2.step()

    mus.append(mu.item())

    if epoch % 100 == 0 or epoch == epochs-1:
        print(f"[epoch:5], {loss.detach().numpy():.9f}")
        loss_values = np.append(loss_values, loss.detach().numpy())
        L_values = np.append(L_values, L.detach().numpy())
        l_values = np.append(l_values, l.detach().numpy())

```

此處差別為 data loss 的計算方式

13. 完整程式碼在附件處

問題(b)的具體步驟如下:

1. 定義精確解與偏微分結果

```

def tru(x, y):
    tru = np.exp(x*y)+np.sinh(x)
    return tru

# Select associated x derivatives .....
def trudx(x, y):
    trudx=y*np.exp(x*y)+np.cosh(x)
    return trudx

# Select y derviatives .....
def trudy(x, y):
    trudy=x*np.exp(x*y)
    return trudy

```

2. 定義源項函數 $f(x, y, \mu)$ 並設真實係數值 $k_0 = 1$ 、 $k_1 = 1$


```

k0=1
k1=1
def f(x,y,mu):
    tru = mu[0]*torch.exp(x*y)*(x*x+y*y)+mu[1]*torch.sinh(x)
    return tru

```

3. 定義一個全連接神經網絡

```

class FCN(nn.Module):
    "Defines a standard fully-connected network in PyTorch"

    def __init__(self, N_INPUT, N_OUTPUT, N_HIDDEN, N_LAYERS):
        super().__init__()
        activation = nn.Tanh # 激活函數
        self.fcs = nn.Sequential(*[nn.Linear(N_INPUT, N_HIDDEN), activation()])
        # 重複 N_LAYERS - 1 次, 以創建多個隱藏層
        self.fch = nn.Sequential(*[nn.Linear(N_HIDDEN, N_HIDDEN), activation()] for _ in range(N_LAYERS-1))
        self.fce = nn.Linear(N_HIDDEN, N_OUTPUT)

    def forward(self, x):
        x = self.fcs(x)
        x = self.fch(x)
        x = self.fce(x)
        return x

```

4. 生成 $x = 0$ 、 $x = 1$ 、 $y = 0$ 、 $y = 1$ 各 30 個隨機邊界資料點，並帶入精確解求 Dirichlet BC、Neumann BC。

```

for i, j in zip(range(n_bc), [0., 1., 0, 1.]):
    points = (engine.random(n=n_data_per_bc)[: , 0] - 0.) * 1
    #points = np.linspace(0, +1, n_data_per_bc)
    if i < 2:
        data[i, :, 0] = j
        data[i, :, 1] = points
    else:
        data[i, :, 0] = points
        data[i, :, 1] = j

```

```

for j in range(0,n_data_per_bc):
    # bord x = 0
    # 計算Dirichlet problem 邊界問題
    data[0, j, 2] = tru(data[0, j, 0], data[0, j, 1])
    # 計算Neumann problems 邊界問題 (導數)
    data[0, j, 3] = trudx(data[0, j, 0], data[0, j, 1])
    data[0, j, 4] = trudy(data[0, j, 0], data[0, j, 1])

    # bord x = 1
    data[1, j, 2] = tru(data[1, j, 0], data[1, j, 1])
    data[1, j, 3] = trudx(data[1, j, 0], data[1, j, 1])
    data[1, j, 4] = trudy(data[1, j, 0], data[1, j, 1])

    # bord y = 0
    data[2, j, 2] = tru(data[2, j, 0], data[2, j, 1])
    data[2, j, 3] = trudx(data[2, j, 0], data[2, j, 1])
    data[2, j, 4] = trudy(data[2, j, 0], data[2, j, 1])

    # bord y = 1
    data[3, j, 2] = tru(data[3, j, 0], data[3, j, 1])
    data[3, j, 3] = trudx(data[3, j, 0], data[3, j, 1])
    data[3, j, 4] = trudy(data[3, j, 0], data[3, j, 1])

```

5. 生成 $0 \leq x \leq 1$ 、 $0 \leq y \leq 1$ 的 500 個內部資料點，並儲存。

```

Nc = 500
engine = qmc.LatinHypercube(d=2)
colloc = engine.random(n=Nc)
colloc = 1 * (colloc - 0)

```

6. 創建一個神經網路，包括 2 個輸入、1 個輸出、6 層隱藏層，每層隱藏層有 25 個神經元。

```

pinn = FCN(2,1,25,6)

```

7. 定義 PINN 輸出與偏微分結果

```

def u_nn(x, y):
    u = pinn(torch.cat([x, y], dim=1)) #排成[[x0,y0],
                                         #      [x1,y1]]
    return u

def uderx(x, y):
    u = pinn(torch.cat([x, y], dim=1))
    uderx = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
    return uderx

def uderx(x, y):
    u = pinn(torch.cat([x, y], dim=1))
    uderx = torch.autograd.grad(u, y, torch.ones_like(u), create_graph=True)[0]
    return uderx

```

8. 定義 Poisson equations，並對輸出做均方誤差

```

def F_nn(x, y, mu):
    u0 = u_nn(x,y)
    u_x = torch.autograd.grad(u0, x, torch.ones_like(u0), create_graph=True)[0]
    u_y = torch.autograd.grad(u0, y, torch.ones_like(u0), create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    u_yy = torch.autograd.grad(u_y, y, torch.ones_like(u_y), create_graph=True)[0]
    # Select the residual equation ....
    F = u_xx + u_yy - f(x,y,mu)
    retour = torch.mean(F ** 2)
    return retour

```

9. 定義均方誤差

```

def mse(y, y_):
    return torch.mean((y-y_)**2)

```

10. 定義了兩個可學習的參數 μ_0 、 μ_1 (標量), 用於估計原項函數中的真實係數值 k_0 、 k_1 。

```

mu = torch.nn.Parameter(torch.zeros(2, requires_grad=True))

```

11. 設定神經網路的優化器為 Adam 和學習率為 2×10^{-4} 、 10^{-1}

```

optimiser1 = torch.optim.Adam(pinn.parameters(), lr=1e-4)
optimiser2 = torch.optim.Adam([mu], lr=1e-1)

```

12. 執行 80000 的迭代訓練

Dirichlet BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_rnn(x_c, y_c, mu)
    #loss on data
    T_ = u_rnn(x_d, y_d)
    l = mse(t_d, T_)

    loss = lambda1*L + lambda2*l
    #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    loss.backward()
    #使用優化器的step()方法更新模型參數
    optimiser1.step()
    optimiser2.step()
    mus = np.append(mus, np.array([[mu[0].item(), mu[1].item()]]), axis=0)
    if epoch % 100 == 0 or epoch == epochs-1:
        print(f"epoch:5, {loss.detach().numpy():.9f}")
        loss_values = np.append(loss_values, loss.detach().numpy())
        L_values = np.append(L_values, L.detach().numpy())
        l_values = np.append(l_values, l.detach().numpy())

```

Neumann BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_rnn(x_c, y_c, mu)
    #loss on data
    T_ = uderx(x_d, y_d)
    l = mse(t_dx, T_)
    T_ = uderx(x_d, y_d)
    l += mse(t_dy, T_)

    loss = lambda1*L + lambda2*l
    #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    loss.backward()
    #使用優化器的step()方法更新模型參數
    optimiser1.step()
    optimiser2.step()
    mus = np.append(mus, np.array([[mu[0].item(), mu[1].item()]]), axis=0)
    if epoch % 100 == 0 or epoch == epochs-1:
        print(f"epoch:5, {loss.detach().numpy():.9f}")
        loss_values = np.append(loss_values, loss.detach().numpy())
        L_values = np.append(L_values, L.detach().numpy())
        l_values = np.append(l_values, l.detach().numpy())

```

Robin BC:

```

for epoch in range(epochs):
    optimiser1.zero_grad() #將梯度清零 防止梯度值累加
    optimiser2.zero_grad()
    #loss on PDE
    L = F_nn(x_c, y_c, mu)
    #loss on data
    T_ = u_nn(x_d, y_d)
    l = mse(t_d, T_)
    T_ = uderx(x_d, y_d)
    l += mse(t_dx, T_)
    T_ = uderx(x_d, y_d)
    l += mse(t_dy, T_)

    loss = lambda1*L + lambda2*l
    #計算加權總損失loss, 對其進行反向傳播來獲得所有參數的梯度
    loss.backward()
    #使用優化器的step()方法更新模型參數
    optimiser1.step()
    optimiser2.step()
    mu = np.append(mu, np.array([[mu[0].item(), mu[1].item()]]), axis=0)
    if epoch % 100 == 0 or epoch == epochs-1:
        print(f"epoch:5, {loss.detach().numpy():.9f}")
        loss_values = np.append(loss_values, loss.detach().numpy())
        L_values = np.append(L_values, L.detach().numpy())
        l_values = np.append(l_values, l.detach().numpy())

```

13. 完整程式碼在附件處

3. 無限位能量子阱

無限位能量子阱的薛丁格方程式如下：

$$-\frac{\hbar^2}{2m}\nabla^2\psi(x,y,z) = E\psi(x,y,z)$$

$$E = \frac{\hbar^2\pi^2}{2m}\left(\frac{n_x^2}{L_x^2} + \frac{n_y^2}{L_y^2} + \frac{n_z^2}{L_z^2}\right)$$

當 x, y, z 座標位於立方體之邊界上時，根據邊界條件所定義，波函數的量值為0，邊界條件以及波函數的解可用以下數學式來表示

$$\psi_{n_x, n_y, n_z}(x, y, z) = A \sin\left(\frac{n_x\pi x}{L_x}\right) \sin\left(\frac{n_y\pi y}{L_y}\right) \sin\left(\frac{n_z\pi z}{L_z}\right)$$

$$A = \sqrt{\frac{8}{L_x \cdot L_y \cdot L_z}}$$

$$V(x, y, z) = 0 \quad x, y, z \in \Omega$$

$$V(x, y, z) = \infty \quad x, y, z \notin \Omega$$

$$\psi(0, y, z) = \psi(L_x, y, z) = 0$$

$$\psi(x, 0, z) = \psi(x, L_y, z) = 0$$

$$\psi(x, y, 0) = \psi(x, y, L_z) = 0$$

其中 n_x 、 n_y 、 n_z 為量子數， L_x 、 L_y 、 L_z 為立方體之邊長。

有鑑於普朗克常數與電子質量的數值過小，可能會在程式計算時產生錯誤以及失去精確性，於是我們將 PDE 與解析解的方程式去單位化，如下所示：

$$x = \frac{x'}{L_x} \quad y = \frac{y'}{L_y} \quad z = \frac{z'}{L_z}$$

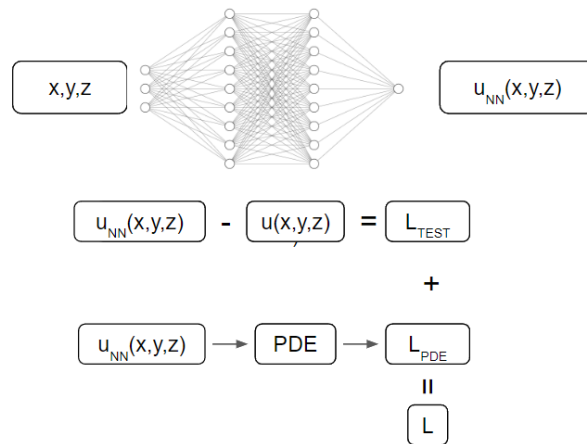
$$\psi_{n_x, n_y, n_z}(x, y, z) = A \sin\left(\frac{n_x \pi x}{L_x}\right) \sin\left(\frac{n_y \pi y}{L_y}\right) \sin\left(\frac{n_z \pi z}{L_z}\right)$$

$$\nabla^2 \psi + E' \psi = 0$$

$$E' = 3\pi^2 \quad A = 2\sqrt{2}$$

其中 x' 、 y' 、 z' 代表未去單位化之原薛丁格方程式的 x 、 y 、 z 。

此命題使用 PyTorch 架構的機器學習來解決，首先定義神經網路的結構，並將其輸出設置為量子阱的波函數。神經網路的輸入是粒子在阱內的座標，而輸出則為對應的波函數值。訓練時為符合量子阱的 Dirichlet 邊界條件，我們直接嵌入一個判斷函式於神經網路的架構中，在對神經網路輸入粒子座標時，函式會以座標判斷粒子是否位在邊界上，將位於邊界之量子所對應之波函數值直接設定為零。



訓練時會用到 PyTorch 中的反向傳播和自動微分函數。損失函數由兩部分組成：一是 $Loss_{PDE}$ ，用來衡量神經網路輸出是否滿足薛丁格方程，二是 $Loss_{TEST}$ ，用來確保神經網路輸出的波函數量值符合理論值。為了最小化兩種損失函數，PINN 會利用梯度下降優化神經網路的權重，使模型逐漸逼近解析解的大小。

具體步驟如下：

1. 定義一個全連接神經網路(使用 PyTorch)

```
class FCN(nn.Module): #Defines a standard fully-connected network in PyTorch

    def __init__(self, N_INPUT, N_OUTPUT, N_HIDDEN, N_LAYERS):
        super(FCN, self).__init__()

        activation = nn.Tanh

        self.fcs = nn.Sequential(
            nn.Linear(N_INPUT, N_HIDDEN), # 全連接層
            activation() # Tanh激活函數
        )

        # 使用列表推導式來定義N_LAYERS - 1個全連接層
        self.fch = nn.Sequential([
            *nn.Sequential(
                nn.Linear(N_HIDDEN, N_HIDDEN), # 全連接層
                activation() # 使用Tanh作為activation函數
            ) for _ in range(N_LAYERS - 1)] # 迴圈執行N_LAYERS - 1次

        # 最後一層全連接層：輸入維度為N_HIDDEN，輸出維度為N_OUTPUT
        self.fce = nn.Linear(N_HIDDEN, N_OUTPUT)

    def forward(self, x):
        # 定義神經網路的前向傳播函數
        x = self.fcs(x) # 第一層全連接層
        x = self.fch(x) # 多層隱藏層全連接層
        x = self.fce(x) # 最後一層全連接層
        return x
```

2. 在神經網路內部加入座標判斷函式

```
def enforce_boundary_conditions(self, x, y, z):
    # 在邊界上強制執行零值
    output = self.forward(torch.cat([x, y, z], dim=1))
    x = x.view(-1, 1)
    y = y.view(-1, 1)
    z = z.view(-1, 1)
    tolerance = 5e-40
    condi_x = torch.isclose(x, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(x, torch.tensor([1], dtype=torch.float64), atol=tolerance)
    condi_y = torch.isclose(y, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(y, torch.tensor([1], dtype=torch.float64), atol=tolerance)
    condi_z = torch.isclose(z, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(z, torch.tensor([1], dtype=torch.float64), atol=tolerance)
    output = torch.where(condi_x, torch.tensor([0], dtype=torch.float64), output)
    output = torch.where(condi_y, torch.tensor([0], dtype=torch.float64), output)
    output = torch.where(condi_z, torch.tensor([0], dtype=torch.float64), output)
    return output
```

3. 生成訓練資料(分為邊界點、隨機點與均勻分布點)

```
def generate_boundary_points(side_points):
    # 生成各邊點數的間隔
    uniform_points = torch.linspace(0, 1, side_points).view(-1, 1)
    zero_points = torch.zeros(side_points, 1)
    one_points = torch.ones(side_points, 1)

    boundaryx1 = torch.cat([one_points, uniform_points, uniform_points.view(-1, 1)], dim=1)
    boundaryx0 = torch.cat([zero_points, uniform_points, uniform_points.view(-1, 1)], dim=1)
    boundaryy1 = torch.cat([uniform_points, one_points, uniform_points.view(-1, 1)], dim=1)
    boundaryy0 = torch.cat([uniform_points, zero_points, uniform_points.view(-1, 1)], dim=1)
    boundaryz1 = torch.cat([uniform_points, uniform_points, one_points.view(-1, 1)], dim=1)
    boundaryz0 = torch.cat([uniform_points, uniform_points, zero_points.view(-1, 1)], dim=1)

    # 合併所有邊界點
    boundary_points = torch.cat([boundaryx1, boundaryx0, boundaryy0, boundaryy1, boundaryz1, boundaryz0], dim=0)

    return boundary_points
```

```
def generate_uniform_points(num_points): #生成點會是傳入值的三次方
    linspace_points = torch.linspace(0, 1, num_points)
    linspace_pointsX = linspace_points
    linspace_pointsY = linspace_points
    linspace_pointsZ = linspace_points
    meshgrid_points = torch.meshgrid(linspace_pointsX, linspace_pointsY, linspace_pointsZ)
    points = torch.stack(meshgrid_points, dim=-1).reshape(-1, 3)
    return points.to(torch.float64)
```

```
# 定義物理點
t_physics = torch.rand(num_physics, 3).to(torch.float64) #隨機生成(x,y,z)點
t_physics = t_physics.requires_grad_(True)
#t_physics = t_physics|
t_physics_x = t_physics[:, 0].view(-1, 1).requires_grad_(True)
t_physics_y = t_physics[:, 1].view(-1, 1).requires_grad_(True)
t_physics_z = t_physics[:, 2].view(-1, 1).requires_grad_(True)

# 定義邊界點
t_boundary = generate_boundary_points(250)
t_boundary = t_boundary.requires_grad_(True).to(torch.float64)
t_boundary_x = t_boundary[:, 0].view(-1, 1).requires_grad_(True)
t_boundary_y = t_boundary[:, 1].view(-1, 1).requires_grad_(True)
t_boundary_z = t_boundary[:, 2].view(-1, 1).requires_grad_(True)
```

```
# 定義測試點
num_test = 32
t_test = generate_uniform_points(num_test)
t_test.to(torch.float64)
t_test_x = t_test[:, 0].view(-1, 1)
t_test_y = t_test[:, 1].view(-1, 1)
t_test_z = t_test[:, 2].view(-1, 1)
t_test_x = t_test_x.requires_grad_(True)
t_test_y = t_test_y.requires_grad_(True)
t_test_z = t_test_z.requires_grad_(True)

t_total_x = torch.cat([t_boundary_x, t_physics_x ], dim=0)
t_total_y = torch.cat([t_boundary_y, t_physics_y ], dim=0)
t_total_z = torch.cat([t_boundary_z, t_physics_z ], dim=0)
t_total_x = t_total_x.requires_grad_(True)
t_total_y = t_total_y.requires_grad_(True)
t_total_z = t_total_z.requires_grad_(True)
```

4. 定義波函數(解析解)

```
def exact_solution(x, y, z, nx=1, ny=1, nz=1):
    x_ = x.view(-1, 1)
    y_ = y.view(-1, 1)
    z_ = z.view(-1, 1)
    ux = ((2)**0.5) * torch.sin(nx*torch.pi*x_)
    uy = ((2)**0.5) * torch.sin(ny*torch.pi*y_)
    uz = ((2)**0.5) * torch.sin(nz*torch.pi*z_)

    tolerance = 5e-40
    Lx = 1
    Ly = 1
    Lz = 1
    condi_x = torch.isclose(x_, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(x_, torch.tensor([Lx], dtype=torch.float64), atol=tolerance)
    condi_y = torch.isclose(y_, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(y_, torch.tensor([Ly], dtype=torch.float64), atol=tolerance)
    condi_z = torch.isclose(z_, torch.tensor([0], dtype=torch.float64), atol=tolerance) | torch.isclose(z_, torch.tensor([Lz], dtype=torch.float64), atol=tolerance)

    ux = torch.where(condi_x, torch.tensor([0], dtype=torch.float64), ux)
    uy = torch.where(condi_y, torch.tensor([0], dtype=torch.float64), uy)
    uz = torch.where(condi_z, torch.tensor([0], dtype=torch.float64), uz)

    u = ux * uy * uz
    return u
```

5. 定義神經網路(三組輸入資料、一組輸出資料)

訓練時由於程式碼過長，所以將參數放在一處統一管理

```
pinn = FCN(3, 1, neurons_per_layer, hidden_layers) #修改輸入維度為3維
pinn = pinn.to(torch.float64) #由於輸入資料皆為雙精度浮點數，所以要把pinn改成適用於double的型態
```

```
#參數調整
hidden_layers = 20
neurons_per_layer = 60
num_physics = 5000
lambda1 = 10
lambda2 = 1
learning_rate = 2e-5
Lx = 1
Ly = 1
Lz = 1
```



```

#參數調整
hidden_layers = 20
neurons_per_layer = 60
num_physics = 5000
num_boundary_per_surface = 200 #實際上點的數量會是這個數值的6倍
lambda1 = 10
lambda2 = 1
learning_rate = 2e-5
Lx = 1
Ly = 1
Lz = 1

```

6. 定義優化器與學習率調節器

```

# 優化器
optimiser = torch.optim.AdamW( pinn.parameters() , lr = learning_rate )
scheduler = torch.optim.lr_scheduler.StepLR(optimiser, step_size=9900, gamma = 10)

```

7. 進行神經網路訓練(反向傳播)

```

for i in range(50001) :
    optimiser.zero_grad()
    # 計算PDE Loss
    u = pinn.enforce_boundary_conditions(t_total_x, t_total_y, t_total_z)
    delta_u = laplacian(u, t_total_x, t_total_y, t_total_z)
    lossP = ( -1 * delta_u - (3*(torch.pi)**2) * u )
    lossP = lossP ** 2
    lossP = torch.mean(lossP)
    tru = exact_solution(t_total_x, t_total_y, t_total_z, nx, ny, nz)
    loss_test = (torch.squeeze(u) - torch.squeeze(tru)) ** 2
    loss_test = torch.mean(loss_test)

    loss = lambda1 * lossP + lambda2 * loss_test
    loss.backward()
    optimiser.step()

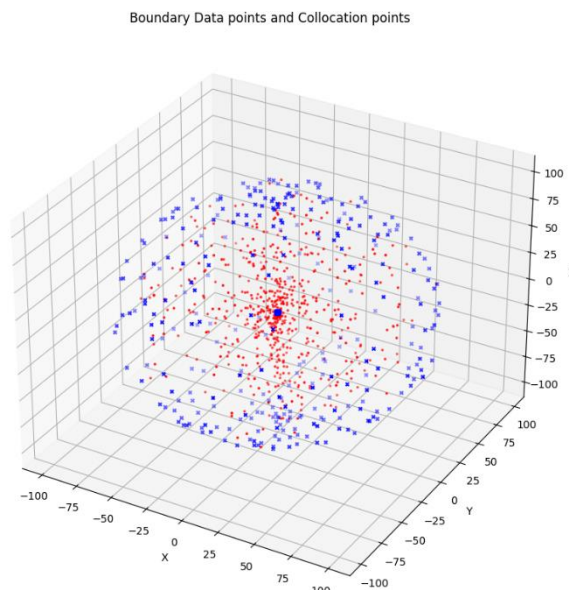
    if i % 100 == 0:
        loss_history.append(loss.item())
        lossP_history.append(lossP.item())
        loss_test_history.append(loss_test.item())

    if i % 5000 == 0:
        print(f'第{i}次迭代', end=' ')
        print(f'loss = {loss.item():.3e}, lossP = {lossP.item():.3e}, loss_test = {loss_test.item():.3e}')
        if i < 10000:
            scheduler.step()
        elif i == 10000:
            print('改為訓練t_test')
            t_total_x = t_test_x
            t_total_y = t_test_y
            t_total_z = t_test_z

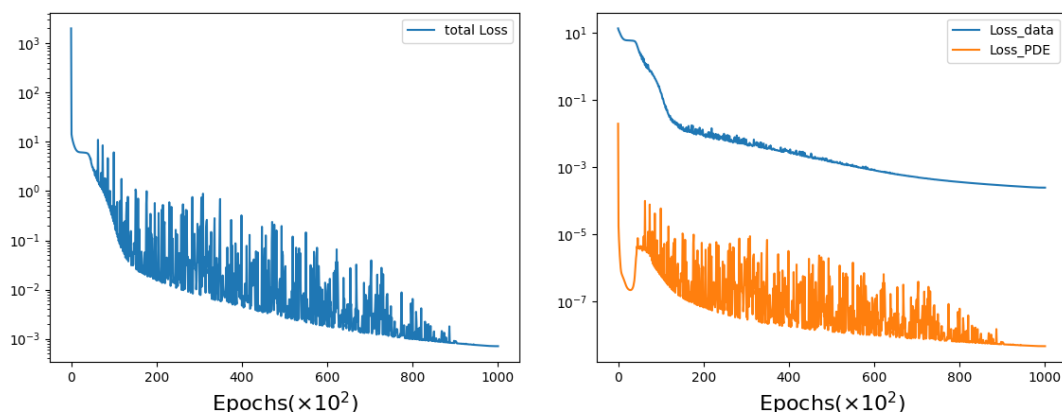
```

結果

1. Laplace 方程



上圖為隨機離散資料點在三維空間中的分布圖，藍色點為半徑等於 1 和 100 的邊界資料點，數量有 600 個，紅色點為半徑介於 1~100 的內部資料點，數量一樣有 600 個。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況，前面步驟 9 有提到，我們拉高了 PDE loss 的計算權重，具體上我們是直接將 PDE loss 乘上 10^5 ，放大 PDE loss，也相對的縮小 data loss 的比重，所以在右圖上可以看到 PDE loss 的收斂值與 data loss 大約就相差了 10^5 倍。

```
99800, 0.000710662
99900, 0.000710652
99999, 0.000710651
computation time: 8406.479
```

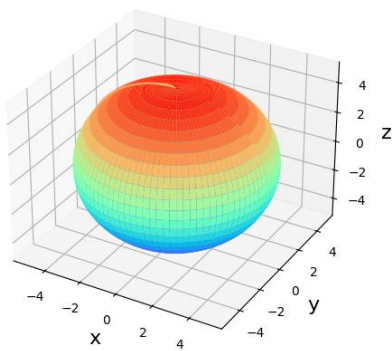
上圖 total loss 最後的收斂值大小約為 7.1×10^{-4} ，共耗時 8406 秒

下圖是我們生成了一個包含 2,000,000 個點的球坐標網格，其中半徑範圍從1~100， θ 範圍從0~ π ， φ 範圍從0~ 2π （網格大小為 $100 \times 100 \times 200$ ）。隨後去計算模型預測解與真實解之間的誤差，並求取所有點上的絕對誤差平均值，數值約為 2.367×10^{-3} 。

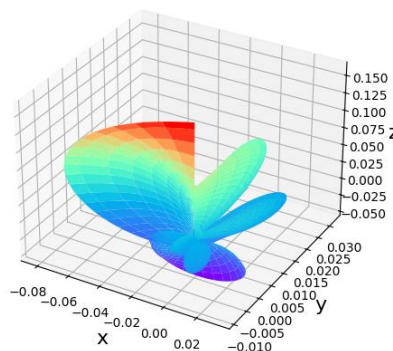
Absolute error: 0.002366866705501394

接著我們在球坐標系下生成了一系列固定半徑 r 的三維圖形，展示了模型預測解與真實解的圖像化差異。具體來說，它對半徑從0~ π 間取5個值，計算並可視化 θ 從0~ π 和 φ 從0~ 2π 的解。此處圖形的半徑改為模型的輸出值，並通過圖形比較模型預測解、真實解以及它們之間的絕對值誤差，來分析模型的準確性。

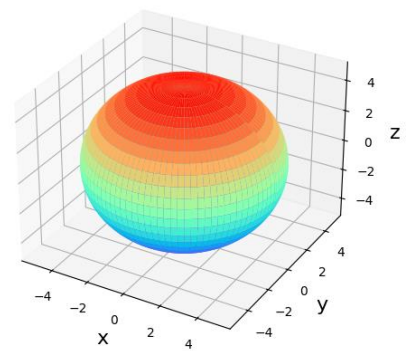
PINN Solution (r=1.0)



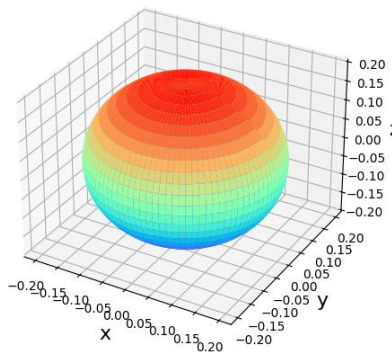
Absolute error (r=1.0)



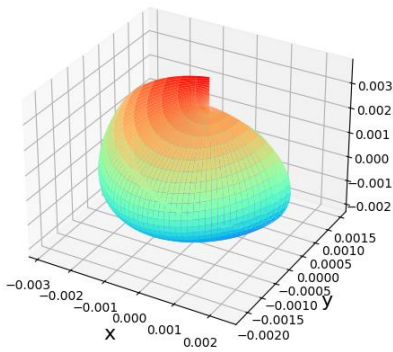
Solution (r=1.0)



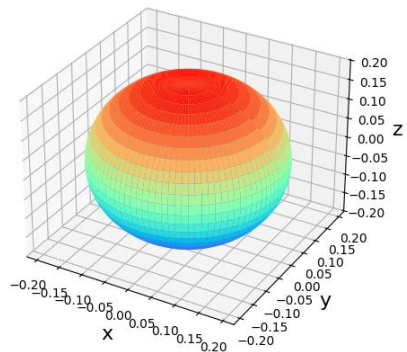
PINN Solution (r=25.75)



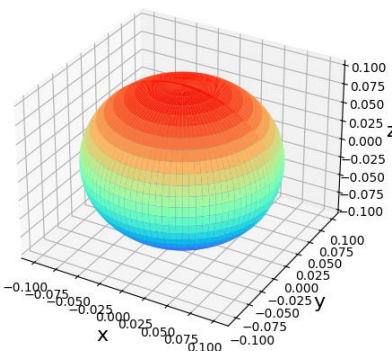
Absolute error (r=25.75)



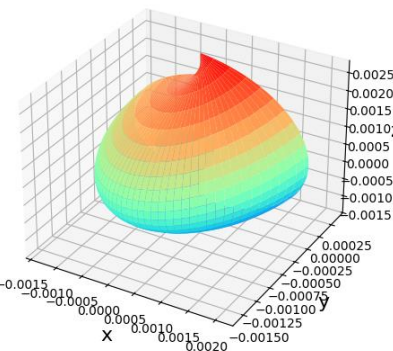
Solution (r=25.75)



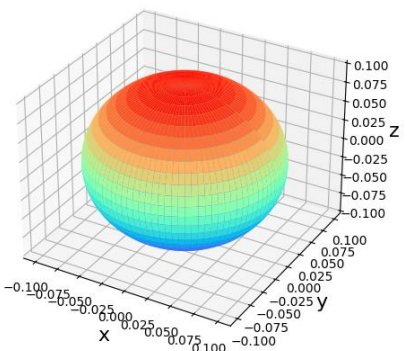
PINN Solution (r=50.5)



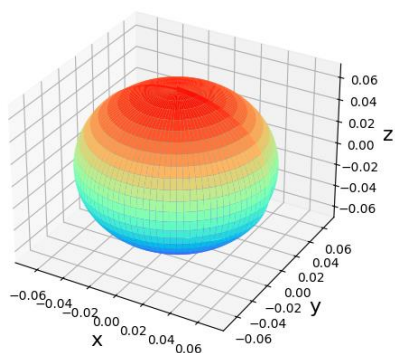
Absolute error (r=50.5)



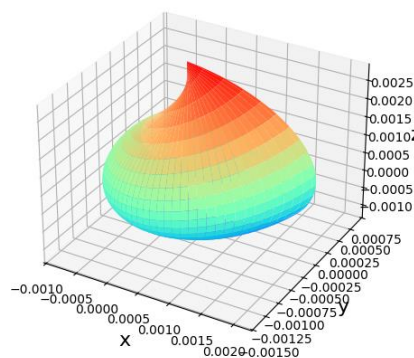
Solution (r=50.5)



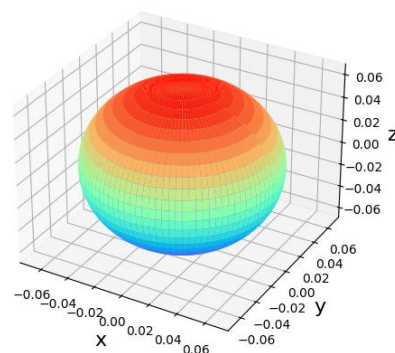
PINN Solution (r=75.25)



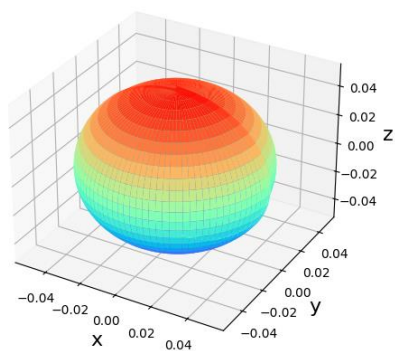
Absolute error (r=75.25)



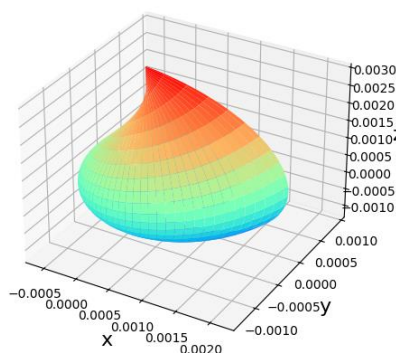
Solution (r=75.25)



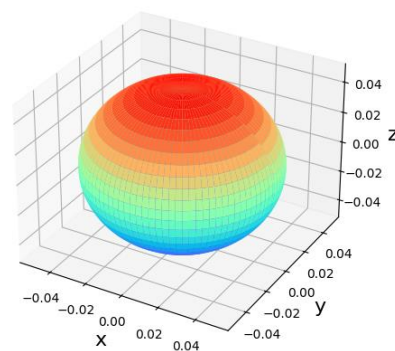
PINN Solution (r=100.0)



Absolute error (r=100.0)



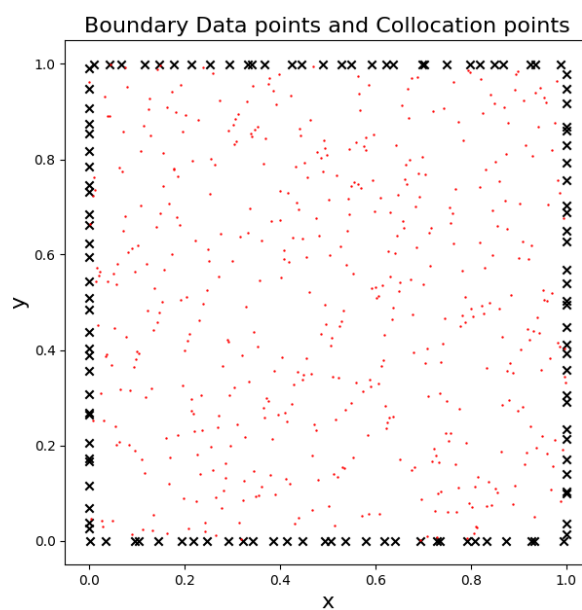
Solution (r=100.0)



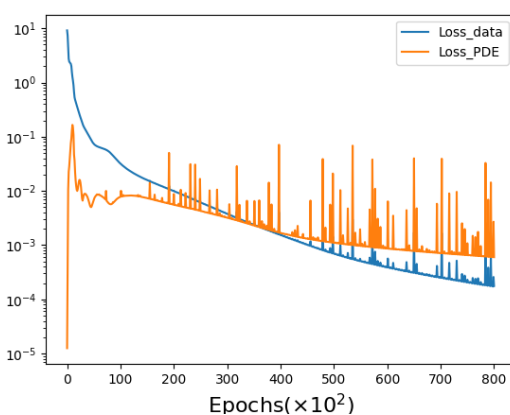
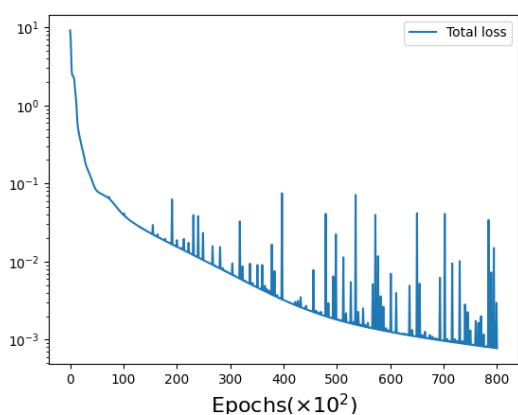
2. Poisson 方程

$$(a) \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - k_0(x^2 + y^2 + 1)e^{x^2+y^2} = 0, \text{ 求 } u(x, y) \text{ 及 } k_0$$

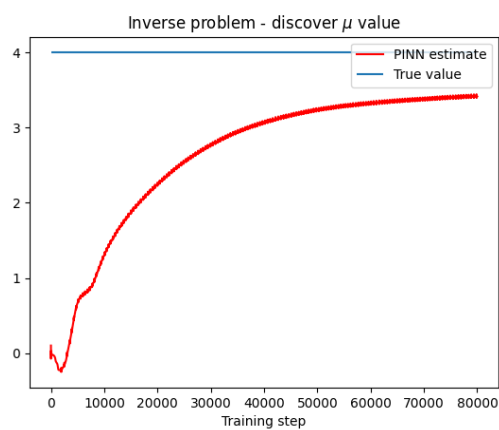
Dirichlet BC



上圖展示了數據集的分佈情況，包括四個邊界處的訓練數據點（黑色叉叉處，共 120 個數據點），以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。

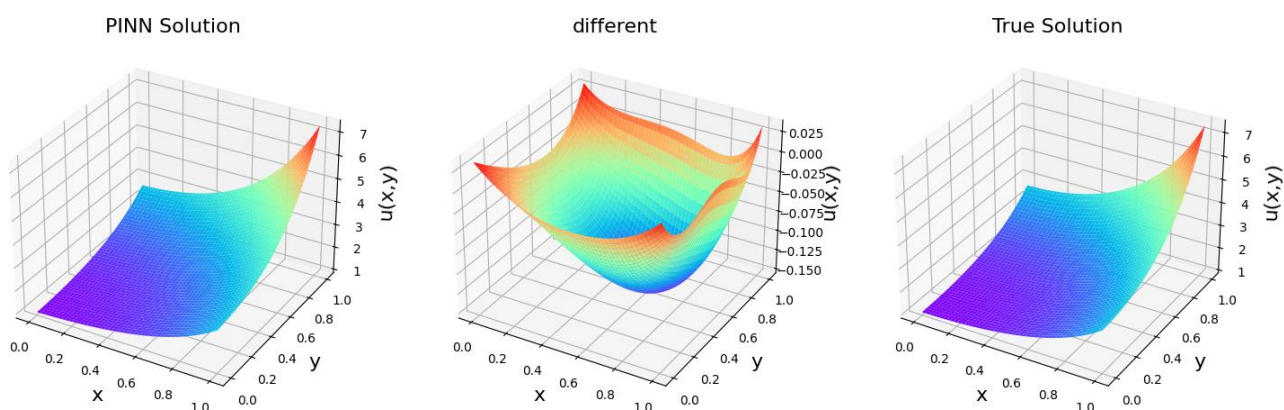


```
79600, 0.000782793
79700, 0.000780737
79800, 0.000778728
79900, 0.002958479
79999, 0.000777414
```

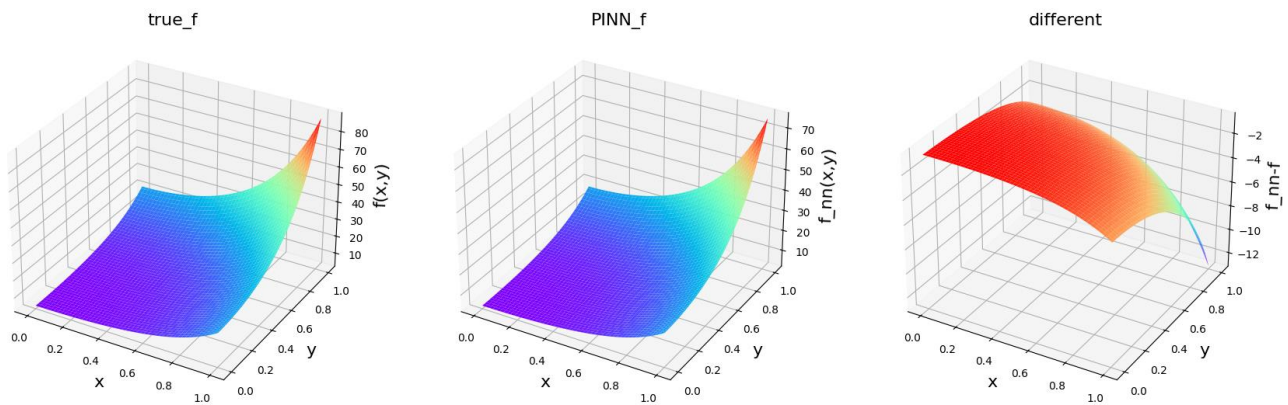
```
computation time: 1695.646
```

```
The final predicted value of mu is =
3.4174423551744044
```

左圖為可學習參數 μ 的收斂情況，紅線為參數 μ ，藍線為真實係數值 k_0 ，右圖則顯示最後 total loss 收斂值為 7.77×10^{-4} ， μ 收斂至3.417，共耗時1695 秒。

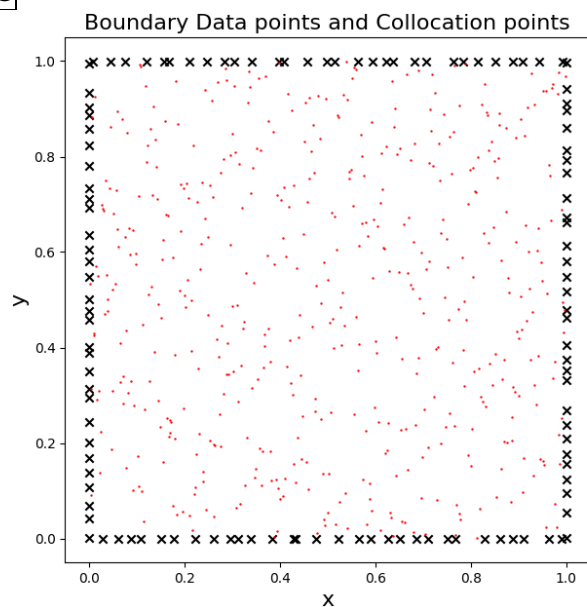


上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從0~1和 y 從0~1的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。

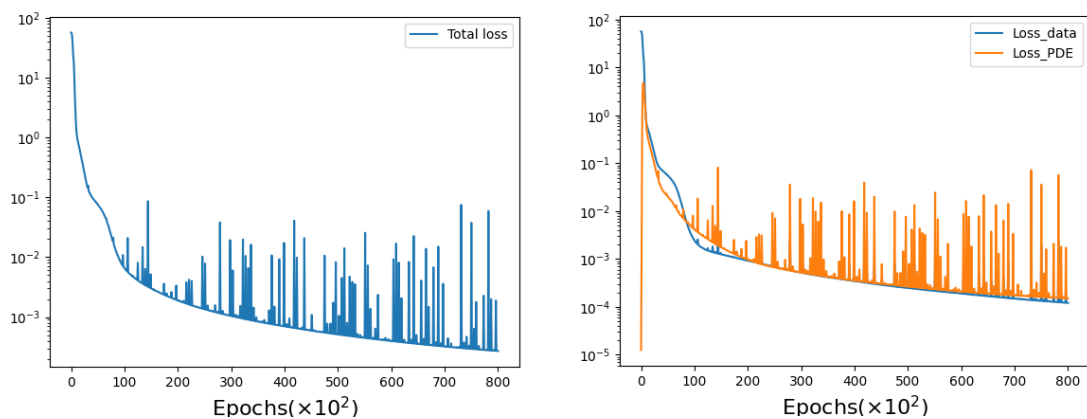


上圖表示 μ 與 k_0 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

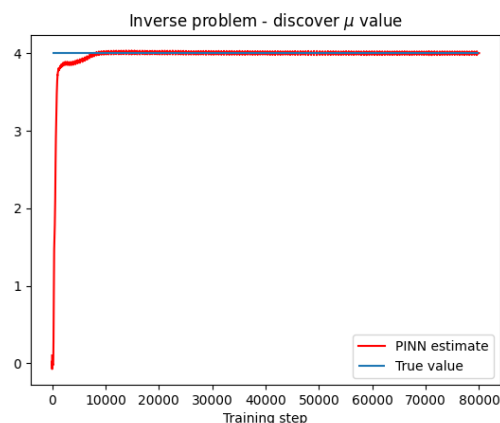
Neumann BC



上圖展示了數據集的分佈情況，包括邊界處的訓練數據點（黑色叉叉處，共 120 個數據點），以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。

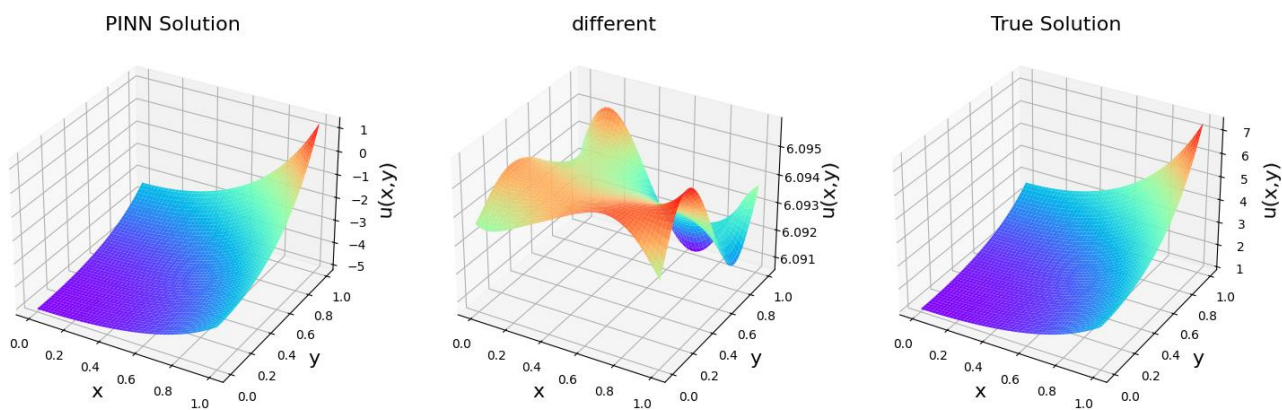


```
79600, 0.001893454
79700, 0.000272106
79800, 0.000271450
79900, 0.000270960
79999, 0.000270459

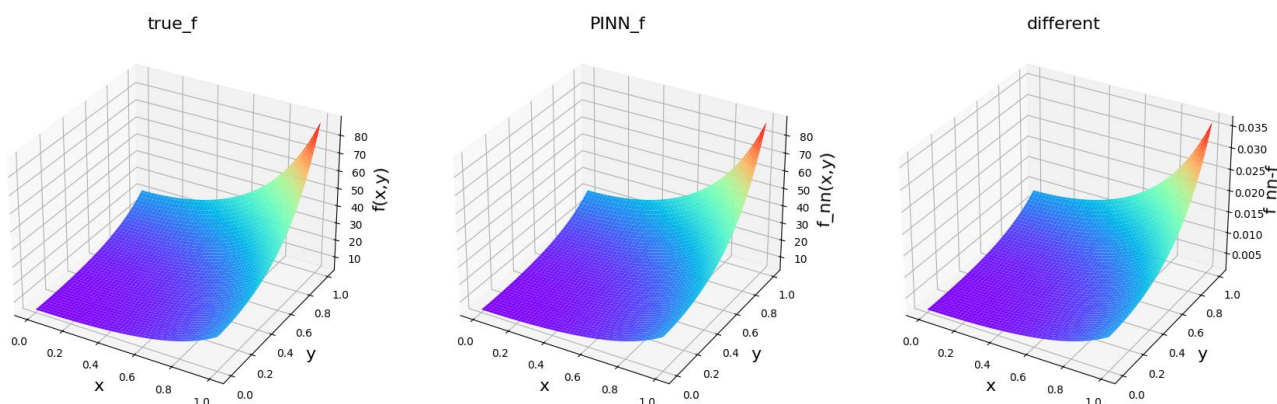
computation time: 1922.537

The final predicted value of mu is =
4.001643488986997
```

左圖為可學習參數 μ 的收斂情況，紅線為參數 μ ，藍線為真實係數值 k_0 ，右圖則顯示最後 total loss 收斂值為 2.7×10^{-4} ， μ 收斂至4.0016，共耗時1922 秒。

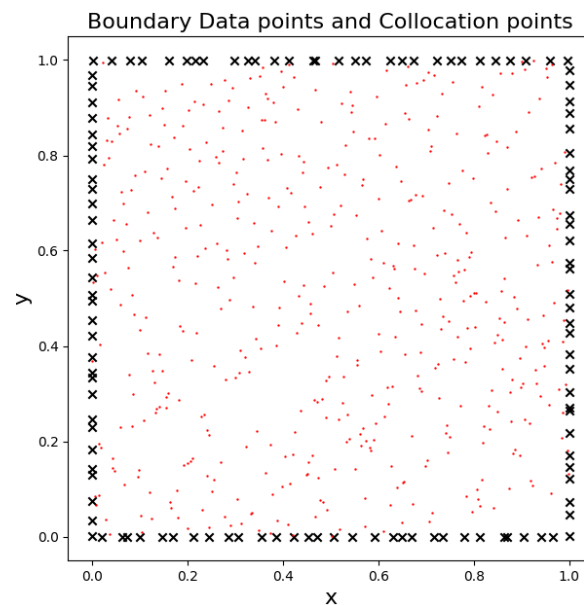


上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從0~1和 y 從0~1的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。

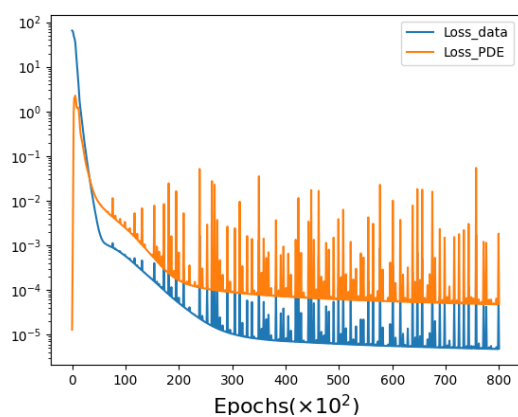
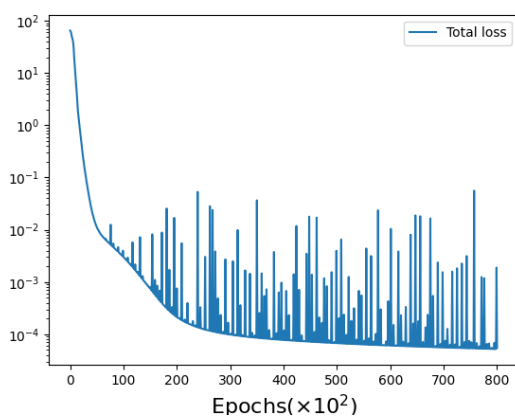


上圖表示 μ 與 k_0 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

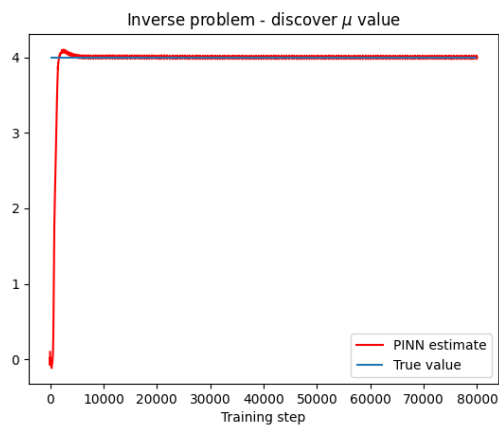
Robin BC



上圖展示了數據集的分佈情況，包括邊界處的訓練數據點（黑色叉叉處，共 120 個數據點），以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。



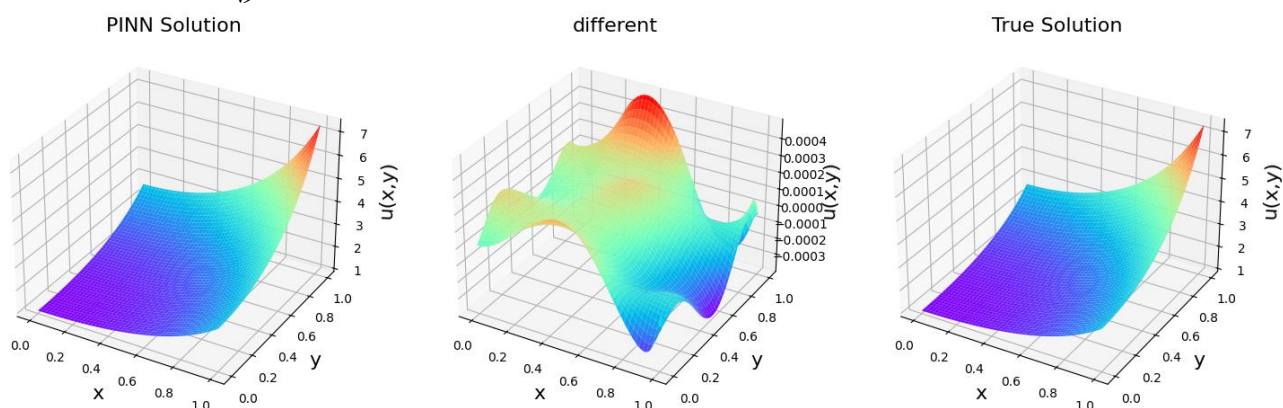
```
79600, 0.000052307
79700, 0.000052234
79800, 0.000052162
79900, 0.001885036
79999, 0.000054917
```

```
computation time: 1995.289
```

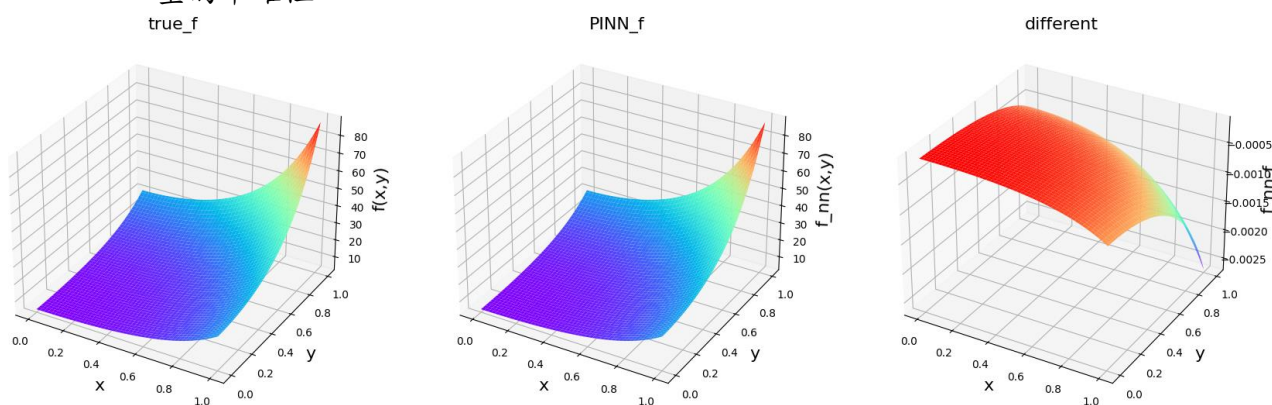
```
The final predicted value of mu is =
3.999880183310621
```

左圖為可學習參數 μ 的收斂情況，紅線為參數 μ ，藍線為真實係數值 k_0 ，右

圖則顯示最後 total loss 收斂值為 5.5×10^{-5} ， μ 收斂至 3.99988，共耗時 1995 秒。



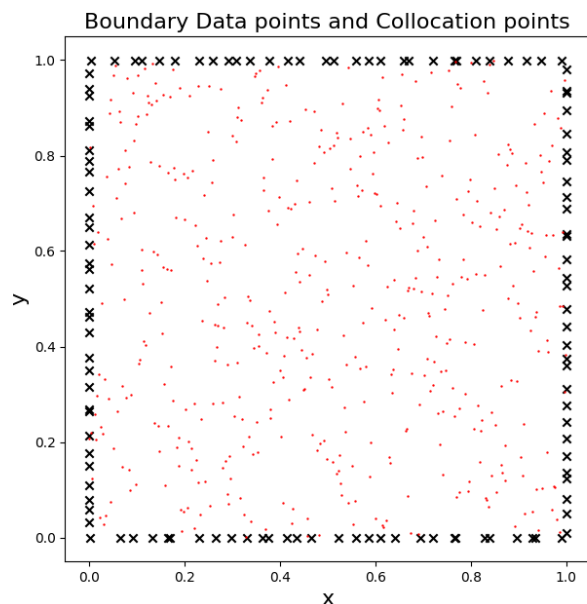
上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從 0~1 和 y 從 0~1 的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。



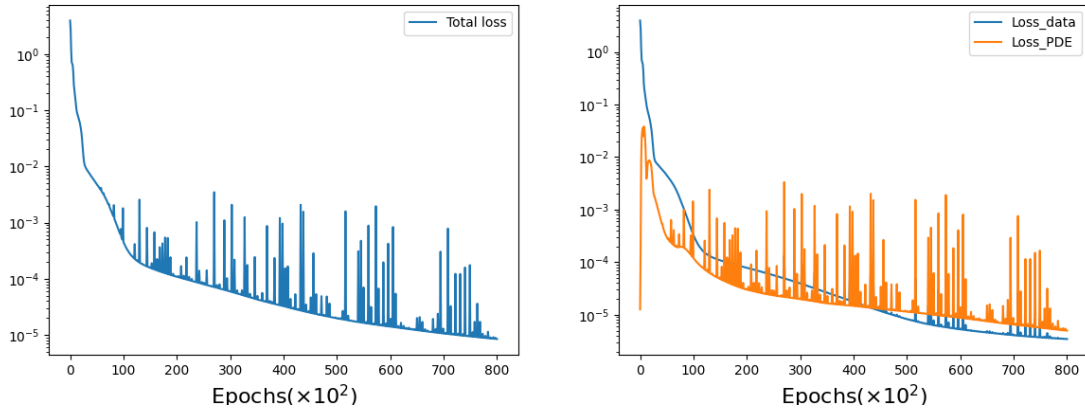
上圖表示 μ 與 k_0 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

(b) $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - k_0 e^{xy}(x^2 + y^2) - k_1 \sinh(x) = 0$ ，求 $u(x,y)$ 、 k_0 、 k_1

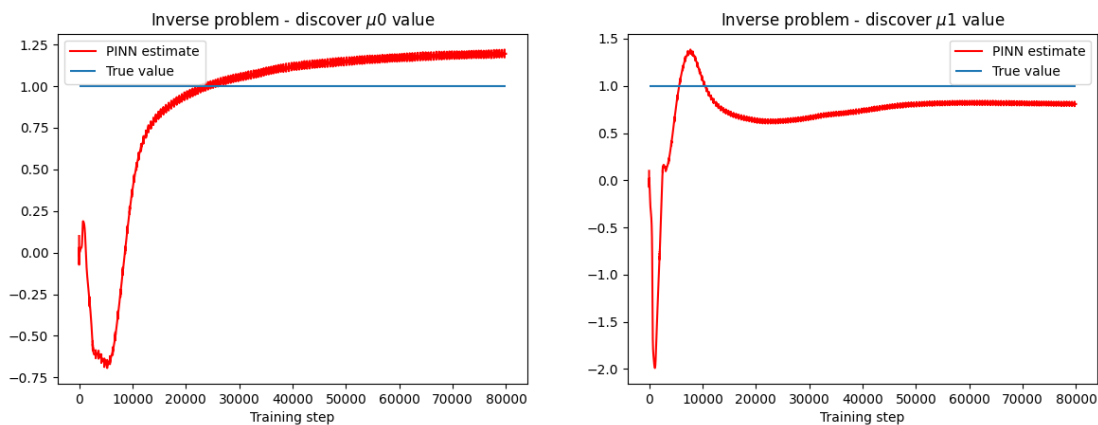
Dirichlet BC



上圖展示了數據集的分佈情況，包括四個邊界處的訓練數據點（黑色叉叉處，共 120 個數據點），以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。



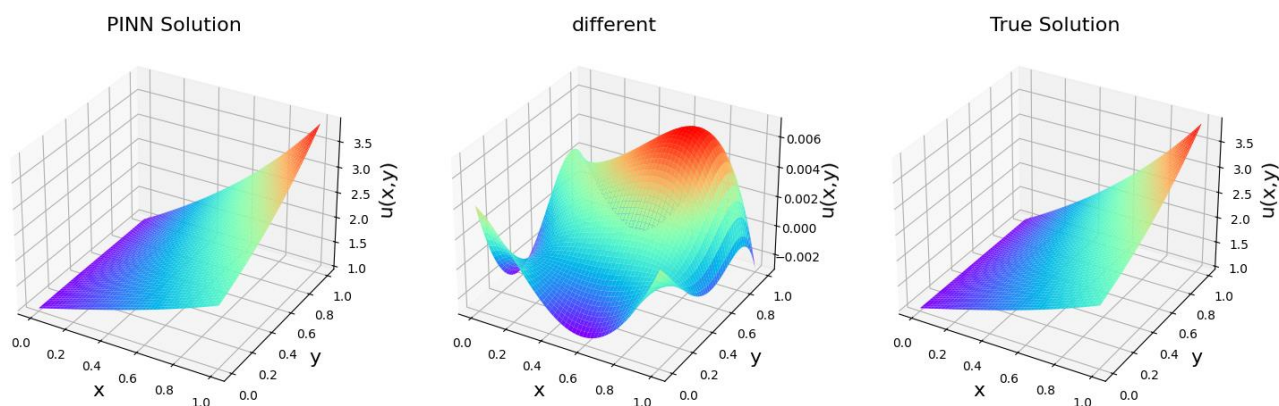
左圖為可學習參數 μ_0 的收斂情況，紅線為參數 μ_0 ，藍線為真實係數值 k_0
右圖為可學習參數 μ_1 的收斂情況，紅線為參數 μ_1 ，藍線為真實係數值 k_1

```
79600, 0.000008526
79700, 0.000008918
79800, 0.000008527
79900, 0.000008486
79999, 0.000008463

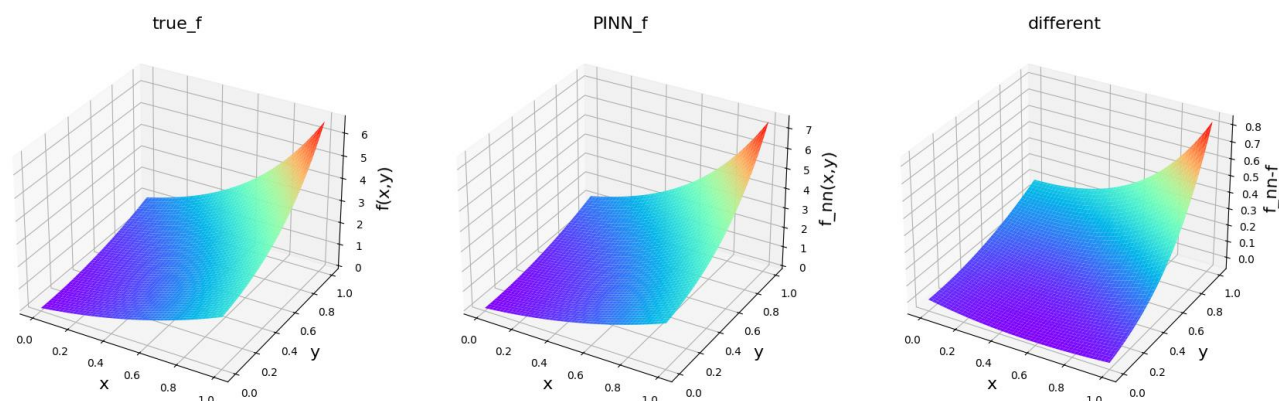
computation time: 1752.527

The final predicted value of mu is =
1.1947058158886295
The final predicted value of mu is =
0.8081223754419287
```

上圖顯示最後 total loss 收斂值為 8.46×10^{-6} ， μ_0 收斂至 1.1947， μ_1 收斂至 0.8081，共耗時 1752 秒。

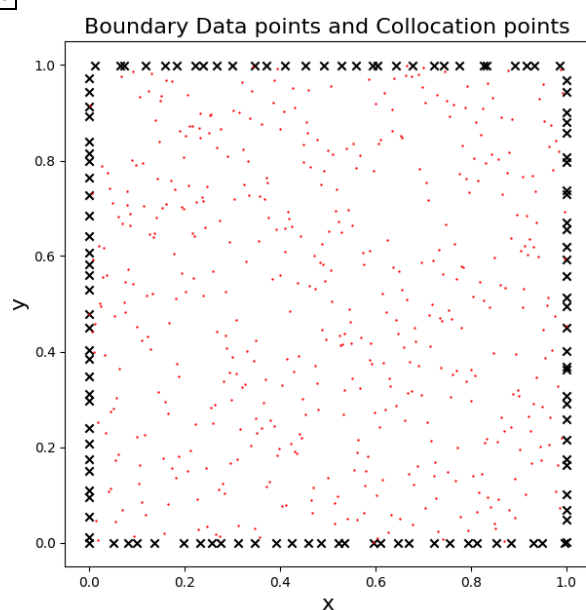


上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從 0~1 和 y 從 0~1 的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。



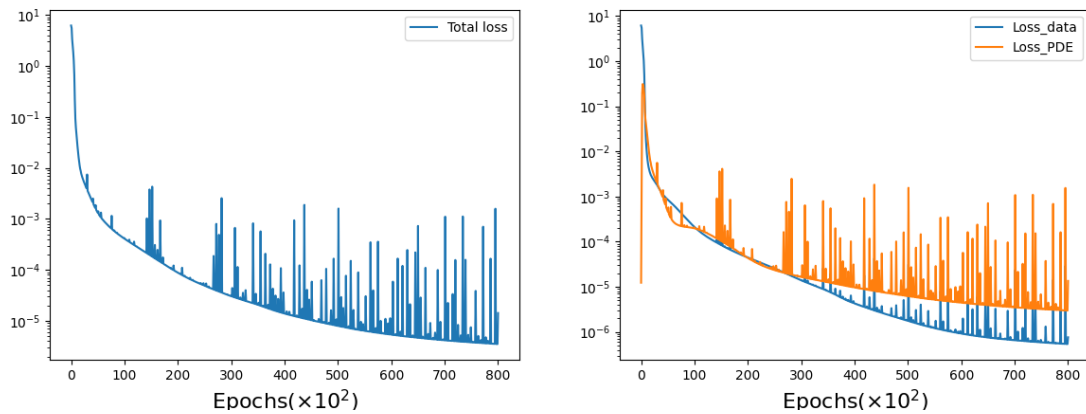
上圖表示 μ_0 、 μ_1 與 k_0 、 k_1 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

Neumann BC

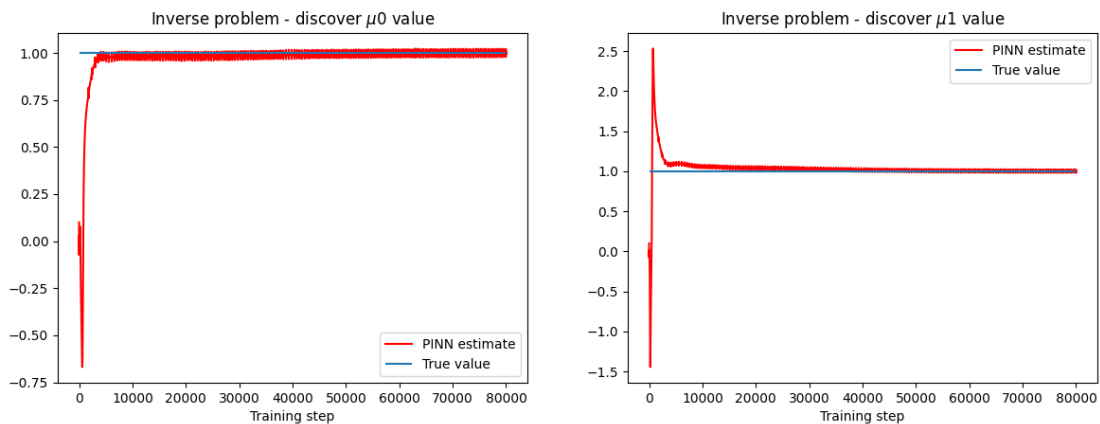


上圖展示了數據集的分佈情況，包括邊界處的訓練數據點（黑色叉叉處，共

120 個數據點)，以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。



左圖為可學習參數 μ_0 的收斂情況，紅線為參數 μ_0 ，藍線為真實係數值 k_0 。

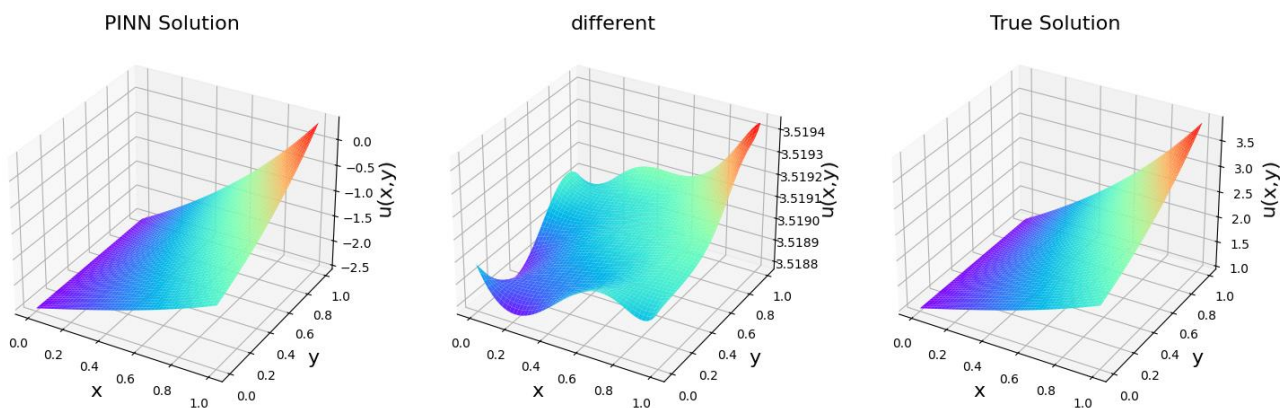
右圖為可學習參數 μ_1 的收斂情況，紅線為參數 μ_1 ，藍線為真實係數值 k_1

```
79600, 0.000003539
79700, 0.000003533
79800, 0.000003527
79900, 0.000003521
79999, 0.000014257

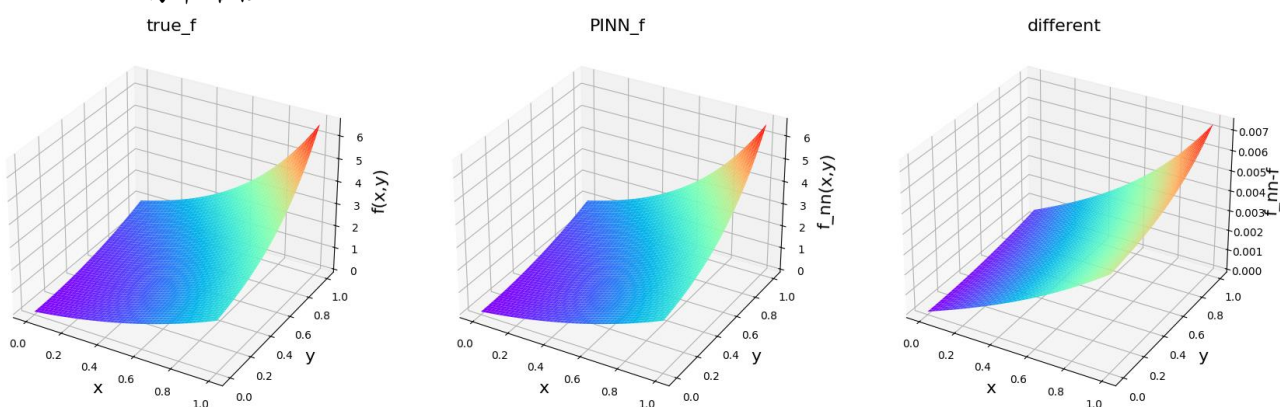
computation time: 1121.953

The final predicted value of mu is =
1.00061915224489
The final predicted value of mu is =
1.0034370290248746
```

上圖顯示最後 total loss 收斂值為 1.4×10^{-5} ， μ_0 收斂至 1.000619， μ_1 收斂至 1.003437，共耗時 1121 秒。

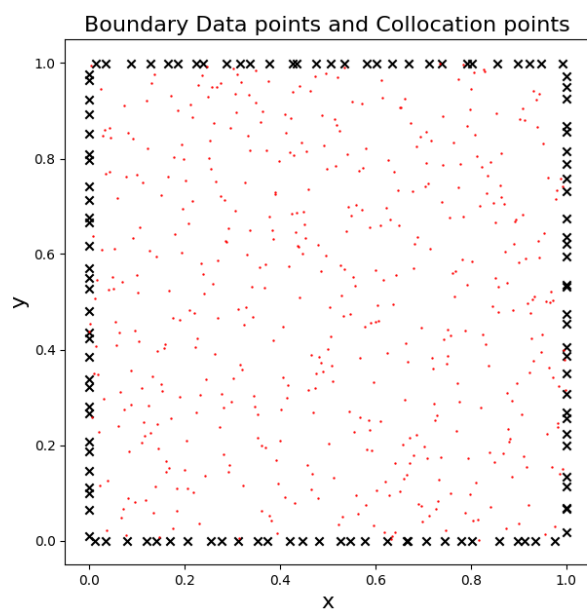


上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從 0~1 和 y 從 0~1 的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。



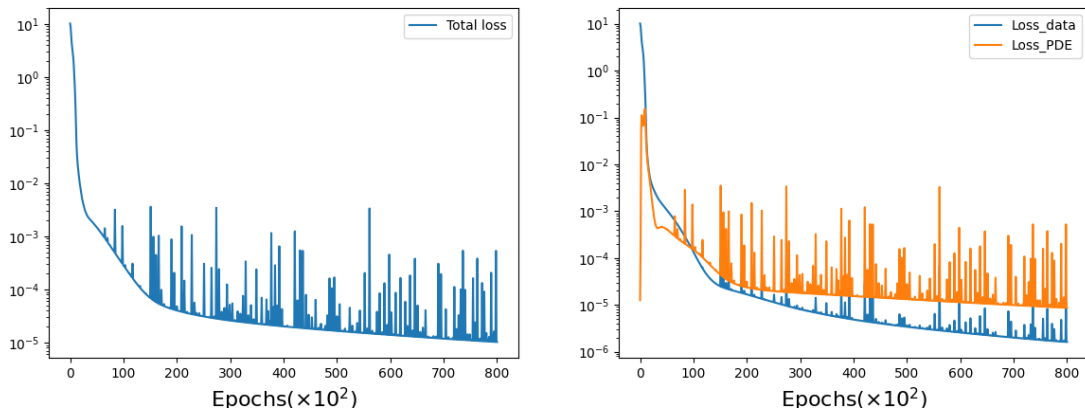
上圖表示 μ_0 、 μ_1 與 k_0 、 k_1 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

Robin BC

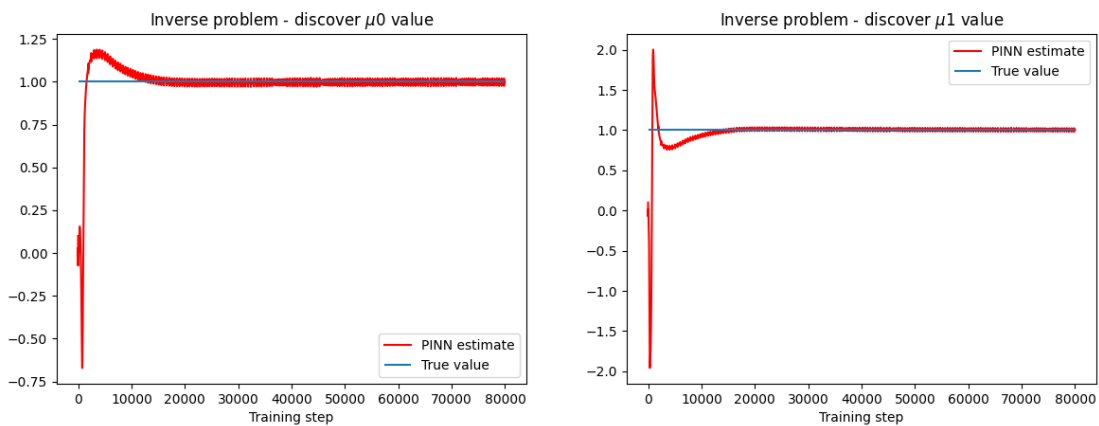


上圖展示了數據集的分佈情況，包括四個邊界處的訓練數據點（黑色叉叉

處，共 120 個數據點），以及範圍內的數據點（紅點部分，共 500 個數據點）。



左圖為隨著訓練次數 total loss 的收斂情形，右圖為 data loss 與 PDE loss 分別的收斂情況。



左圖為可學習參數 μ_0 的收斂情況，紅線為參數 μ_0 ，藍線為真實係數值 k_0 。

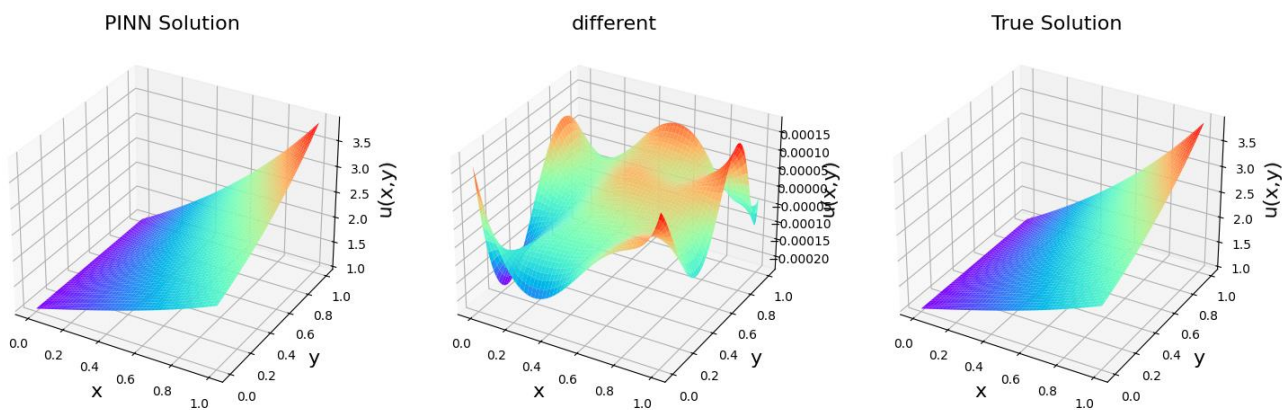
右圖為可學習參數 μ_1 的收斂情況，紅線為參數 μ_1 ，藍線為真實係數值 k_1

```
79600, 0.000010320
79700, 0.000010305
79800, 0.000535516
79900, 0.000010278
79999, 0.000010264

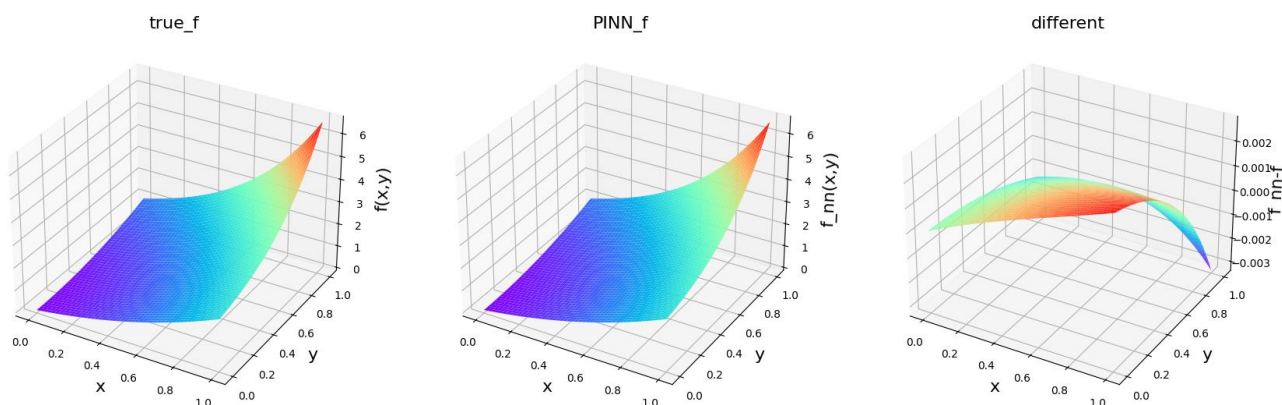
computation time: 2005.722

The final predicted value of mu is =
0.9986223500776746
The final predicted value of mu is =
1.0036108300580389
```

上圖顯示最後 total loss 收斂值為 1.0264×10^{-5} ， μ_0 收斂至 0.9986， μ_1 收斂至 1.00361，共耗時 2005 秒。

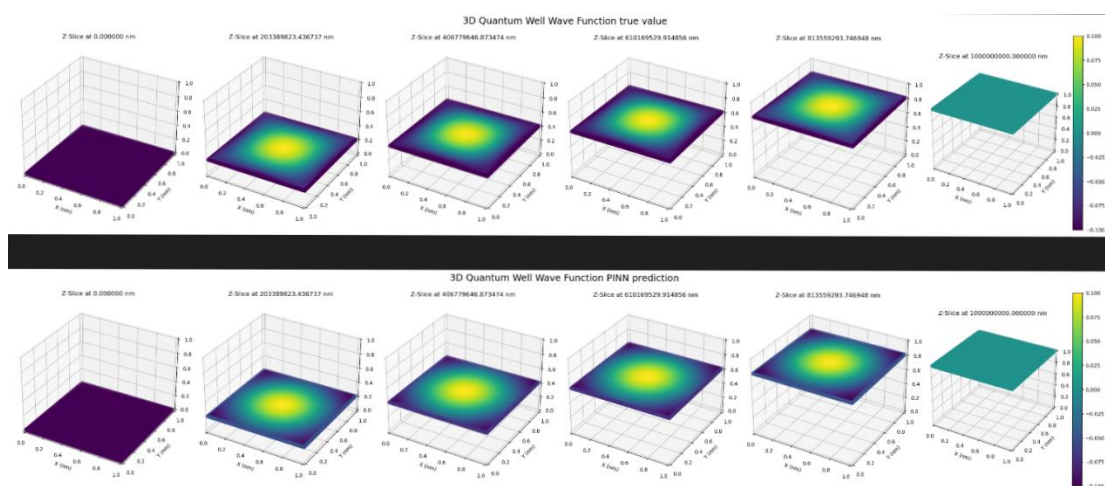


上圖展示了 PINN 與解析解輸出的三維圖，計算並可視化 x 從 0~1 和 y 從 0~1 的解，通過圖形比較模型預測解、真實解以及它們之間的誤差，來分析模型的準確性。

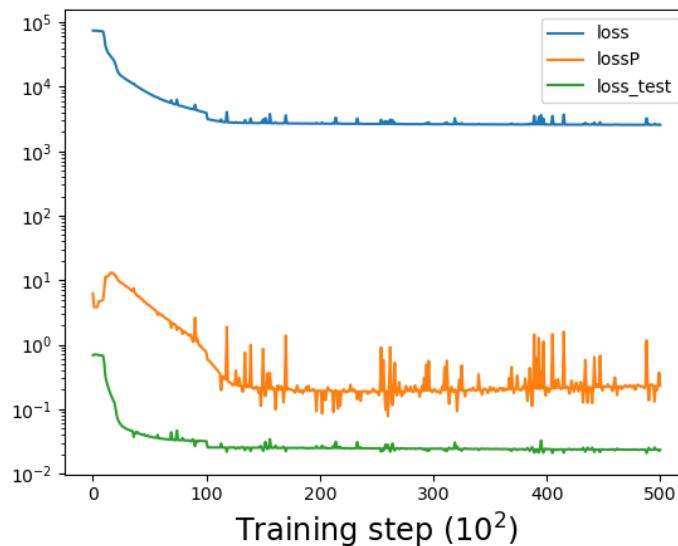


上圖表示 μ_0 、 μ_1 與 k_0 、 k_1 各別帶入源項函數 $f(x,y)$ 的係數項，對應的 $f(x,y)$ 輸出圖，與兩者間的誤差值。

3. 無限位能量子阱



上圖為 PINN 訓練結果與解析解輸出的波函數大小，立方體邊長單位為 1 奈米，由於方程式已去單位化，立方體邊長在程式碼中為 1。由於波函數在座標點上都有波函數量值，不適合直接以實心立方輸出圖像，故沿著 Z 軸分數個切面輸出圖形。



總損失函數為 PDE 損失函數和 Test 損失函數乘上固定比重再相加而成。由上圖可見損失函數 Loss 順利於訓練過程中下降並收斂。

結論與建議

在第一個電磁問題中，由解析解可以知道，此球型導體的電位分布，只與半徑有關，所以在固定半徑的條件下，三維圖會呈現球狀的分布，在畫出解析解的三維圖中也印證了這件事，而觀察訓練結果可以發現到，在 PINN Solution 的圖形上也同樣呈球形分布，Absolute error 的圖形上則發現誤差在最差的情況下也會是輸出數值的 $\frac{1}{10}$ 倍，實際建 2,000,000 個點的球坐標網格去計算的絕對誤差平均值，大約落在 2.367×10^{-3} ，說明 PINN 確時有成功找出半徑在 1~100 間的電位分布，並且訓練的資料量只使用了 600 筆數據(邊界資料)，應證了先前提到 PINN 訓練數據量少的優點。(這裡只算邊界資料數量，是因為訓練中並沒有用到內部資料點的電位值)

在第二個 Poisson 方程問題中，除了求解偏微分方程外，還一同求解了有關源項函數 $f(x, y)$ 的參數反問題，並使用不同的邊界條件做訓練，觀察訓練的結果，可以發現到，在 PINN Solution 三維圖中，不論是(a)或(b)問題，最差表現模型的邊界條件是 Neumann BC，因為只知道函數導數值的關係，所以輸出與解析解始終差了一個常數，其次是 Dirichlet BC，但誤差值相較前著縮小很

多，基本上已經有成功找出方程解，最小的則是 Robin BC，誤差能小至輸出值的 $\frac{1}{1000}$ 倍。

而在找源項函數 $f(x, y)$ 的參數反問題上，最差表現模型的邊界條件為 Dirichlet BC，其次是 Neumann BC，但誤差值已經很小了，最好的則是 Robin BC，為了更直觀看出差距，我們將各自可學習參數 μ 帶回源項函數 $f(x, y)$ 中，並與原係數 k 帶回的 $f(x, y)$ 比較(true_f、PINN_f、different 的圖形)，經過比對差值就可以得到一樣的結論。

在兩項比較下，可以發現 Robin 邊界條件下訓練出的模型，效果最好，並且訓練的資料量只使用了 120 筆數據(邊界資料)，再次應證了先前提到 PINN 訓練數據量少的優點。

在第三個研究項目中，實驗一開始沒有將方程式去單位化，又因方程式常數的量值皆相當小，所以要使用雙精度浮點數的資料型態用以計算，且由於量子阱方程式相當複雜，神經網路的規模也得加大加廣，所以使用 20 層隱藏層，其中每層有 60 個神經元，如此一來就大幅增加了訓練難度，每次訓練都是數十小時起跳，可見 PINN 效力可能不夠用以解決如此複雜的問題上。與指導教授討論後，成功找到方法把方程式去單位化，縮短了很多訓練時間，結果也好上許多。

未來的研究建議可以嘗試將 PINN 應用於更多不同類型的問題，與不同邊界條件上，以驗證其有效性和實用性。

參考文獻

[Hubert Baty] ‘A hands-on introduction to Physics-Informed Neural Networks for solving partial differential equations with benchmark tests taken from astrophysics and plasma physics’. web

[Hubert Baty] python code <https://github.com/hubertbaty/PINNS-PDE.git>

圖片出處 <https://reurl.cc/Mjz2zL>

暑期計畫成果自評

在這次研究中，我們成功運用 Physics-Informed Neural Networks (PINN) 解決了三個物理偏微分方程問題，並與原計畫的目標高度一致。從學術和應用價值角度來看，此研究展示了 PINN 在解決物理偏微分方程和參數反問題中的強大潛力。PINN 不僅在少量數據下能有效訓練，還具備解決未知參數問題的能力，這使其在數值計算、工程應用中具有廣泛的應用潛力。針對未來的應用場景，PINN 可進一步擴展到更複雜的物理問題或不同邊界條件，驗證其更廣泛的應用性。

指導教授評語

我很高興看到這份專題成果報告。學生們在這次研究中展現出了優秀的學習能力和科研素養。以下是我的評語：

這份專題報告展示了學生們對物理信息神經網絡(PINN)的深入理解和應用。他們成功地將 PINN 用於解決三個具有挑戰性的物理問題：三維球坐標中的 Laplace 方程、二維 Poisson 方程及其參數識別、以及三維無限深位能阱的薛丁格方程。特別值得讚賞的是：

1. 學生們不僅掌握了 PINN 的理論基礎，還能靈活地將其應用到不同類型的偏微分方程和邊界條件中。
2. 他們展示了良好的問題分析能力，如在 Poisson 方程中識別未知參數，以及對量子阱問題進行去單位化處理。
3. 報告中呈現的可視化結果和誤差分析非常全面，體現了學生們對結果的深入理解和批判性思考。
4. 學生們注意到了 PINN 在處理複雜問題(如量子阱)時的局限性，這種洞察力對未來研究很有價值。
5. 整個研究過程中，學生們展現出了 excellent 的团队合作能力和解決問題的能力。

未來研究建議：

1. 可以嘗試將 PINN 應用於更複雜的物理系統或非線性問題。
2. 探索如何提高 PINN 在處理高維問題時的效率。
3. 考慮將 PINN 與其他機器學習方法結合，以解決更廣泛的科學計算問題。

整體而言，這是一份非常出色的專題研究報告。學生們不僅達成了預期目標，還展示了深厚的理論基礎和實踐能力。我相信這次研究經歷將為他們未來在科研或工程領域的發展奠定堅實基礎。

附件

程式碼 <https://github.com/raychang1111/PINN-project.git>

<https://colab.research.google.com/drive/1gofiCuuKmlEqbl4GdiJSbzUYySeSZkR?usp=sharing>